```
# /usr/local/bin/qemu-system-x86_64  -monitor stdio \
  -cpu qemu64 \
  -vga cirrus \
  -m 4096  -smp 4   \
  ...
```

This results in a new prompt `(qemu)` in the terminal window as shown in Figure 11.

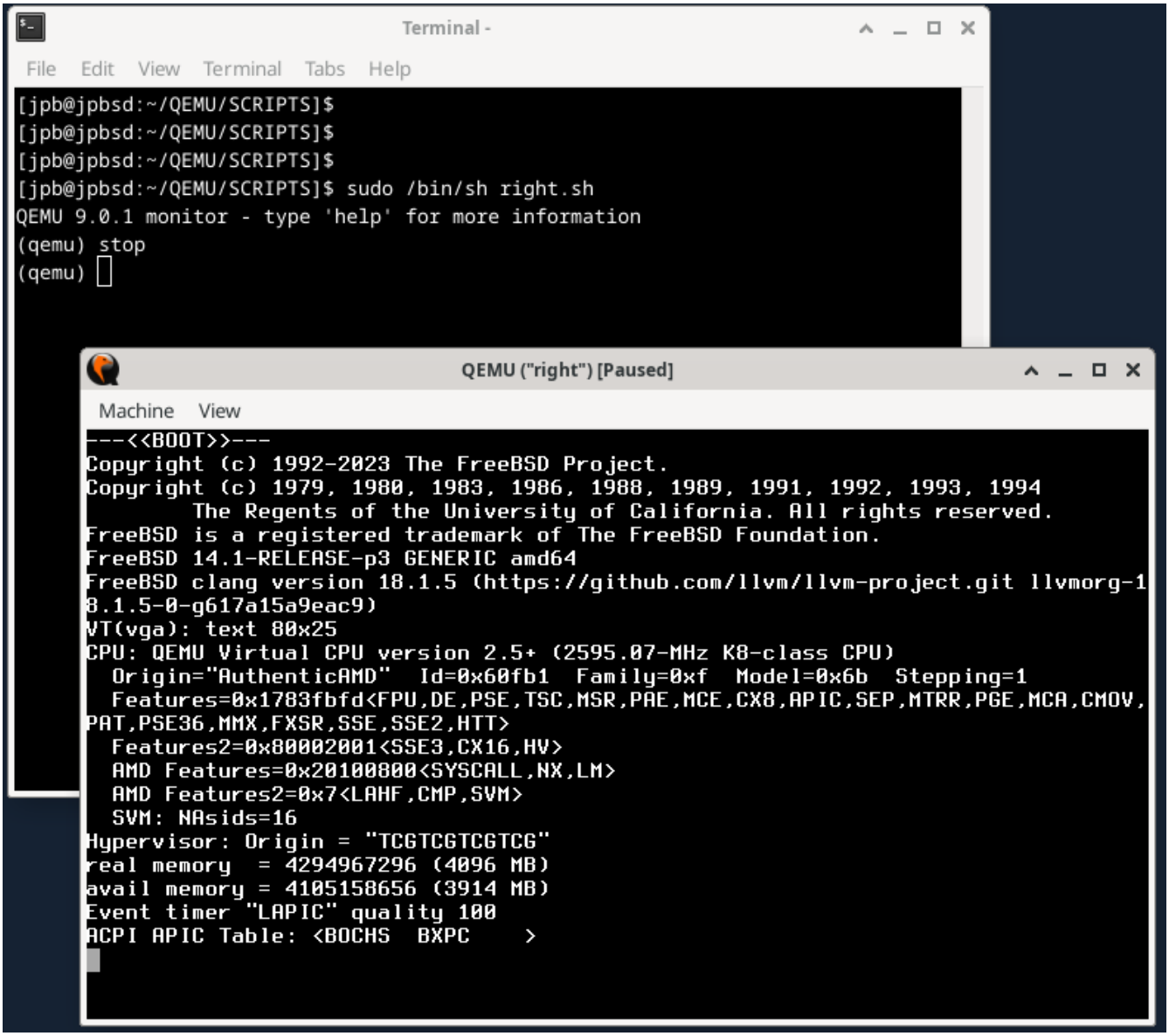



*Figure 70. QEMU Monitor Prompt and "stop" Command*

The image also shows the `stop` command freezing the system during the FreeBSD boot sequence. The system will remain frozen until the `cont` command is entered in the monitor.

#### 24.6.6.1. Adding a New Disk to the VM

To add a new disk to a running VM, the disk needs to be prepared as above:

```
% cd ~/QEMU/VM
```

```
% qemu-img create -f raw  new10G.img  10G
```

Figure 12 shows the monitor command sequence needed to add a new disk in the VM. Once the device has been added with the `device_add` command in the monitor it shows up on the FreeBSD system console shown in the lower part of the figure. The disk can be configured as needed.

Note that the new disk must be added to the startup script if it is to be used after a VM reboot.

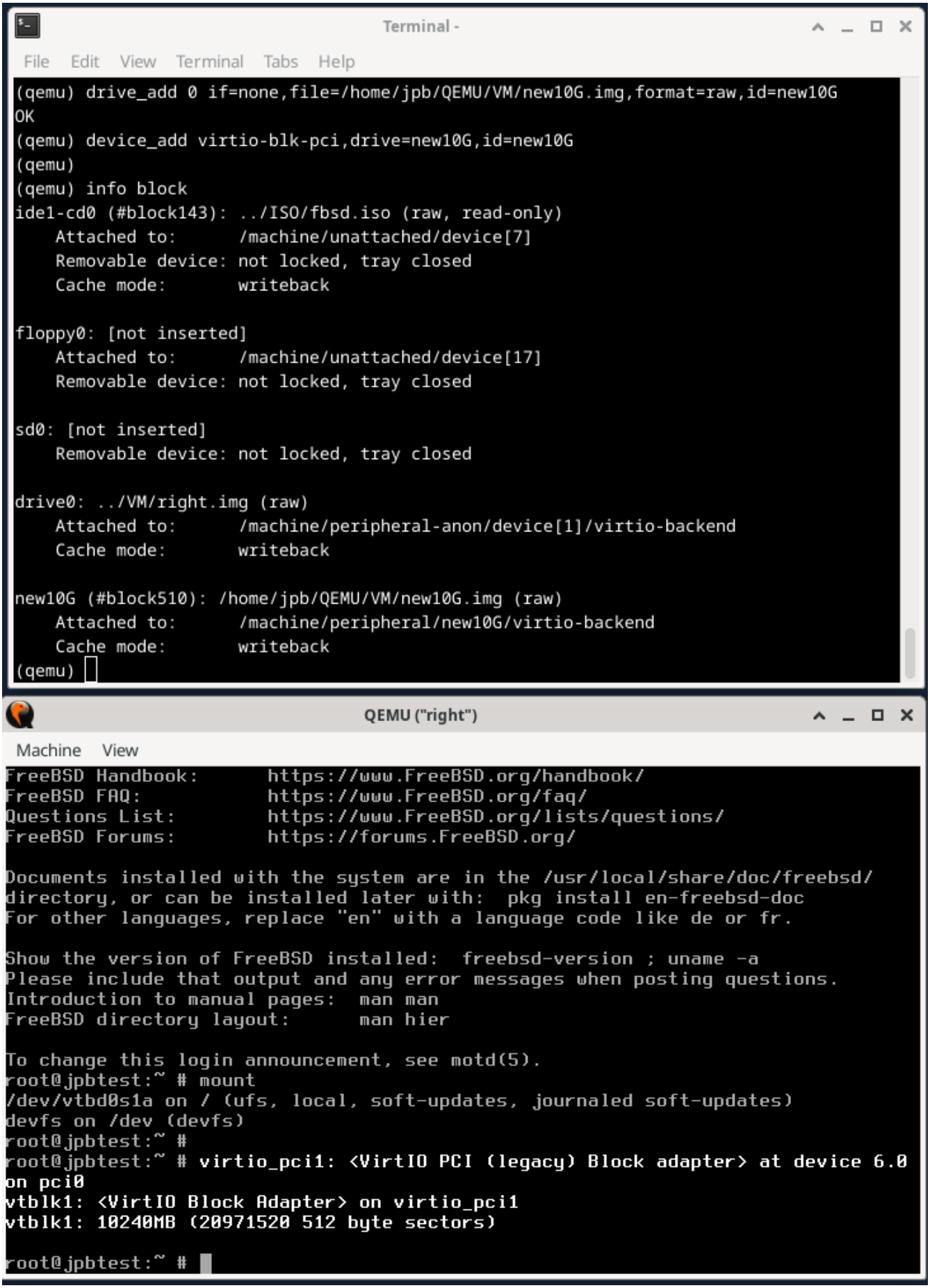



*Figure 71. QEMU Monitor Commands to Add a New Disk*

#### 24.6.6.2. Using the QEMU Monitor to Manage Snapshots

QEMU's documentation describes several similar concepts when using the term **snapshot**. There is the `-snapshot` option on the command line which refers to using a drive or portion of a drive to contain a copy of a device. Then there are the monitor commands `snapshot_blkdev` and `snapshot_blkdev_internal` which describe the actual act of copying the blockdev device. Finally, there are the monitor commands `savevm`, `loadvm`, and `delvm` commands which refer to creating and saving, loading, or deleting a copy of an entire virtual machine. Along with the latter, the monitor `info snapshots` command lists out details of recent snapshots.

This section will focus on creating, saving, and loading a complete VM image and will use the term **snapshot** for this purpose.

To start, recreate the "left" VM from scratch, this time using the `qcow2` format.

```
% cd ~/QEMU/VM
% rm left.img
% qemu-img create -f qcow2 left.qcow2 16G  # Clean file for a new FreeBSD
installation.
% cd ../SCRIPTS
# /bin/sh left.sh                    # See the below program listing.
```

Once the installation is complete, reboot, this time using the `-monitor stdio` option to allow use of the monitor.

```
# left VM script.
/usr/local/bin/qemu-system-x86_64  -monitor stdio \
  -cpu qemu64 \
  -vga std \
  -m 4096 \
  -smp 4   \
  -cdrom ../ISO/fbsd.iso \
  -boot order=cd,menu=on \
  -blockdev driver=file,aio=threads,node-name=imgleft,filename=../VM/left.qcow2 \
  -blockdev driver=qcow2,node-name=drive0,file=imgleft \
  -device virtio-blk-pci,drive=drive0,bootindex=1  \
  -netdev tap,id=nd0,ifname=tap0,script=no,downscript=no,br=bridge0 \
  -device e1000,netdev=nd0,mac=02:20:6c:65:66:74 \
  -name \"left\"
```

To demonstrate snapshots, the following procedure can be used:

1. Install FreeBSD from scratch
2. Prepare the environment and take a snapshot with the `savevm` monitor command
3. Install several packages
4. Shut down the system
5. Restart a bare QEMU instance and utilize the monitor command `loadvm` to restore the VM

6. Observe that the restored VM does not have any packages

During the "Prepare the environment" step, in a separate virtual console (ttyv1), an editing session with vi(1) is initiated simulating user activity. Additional programs may be started if desired. The snapshot should account for the state of all applications running at the time the snapshot is taken.

Figure 13 shows the newly installed FreeBSD system with no packages, and separately, the editing session on ttyv1. The vi(1) editor is currently in `insert` mode with the typist typing the word "broadcast".

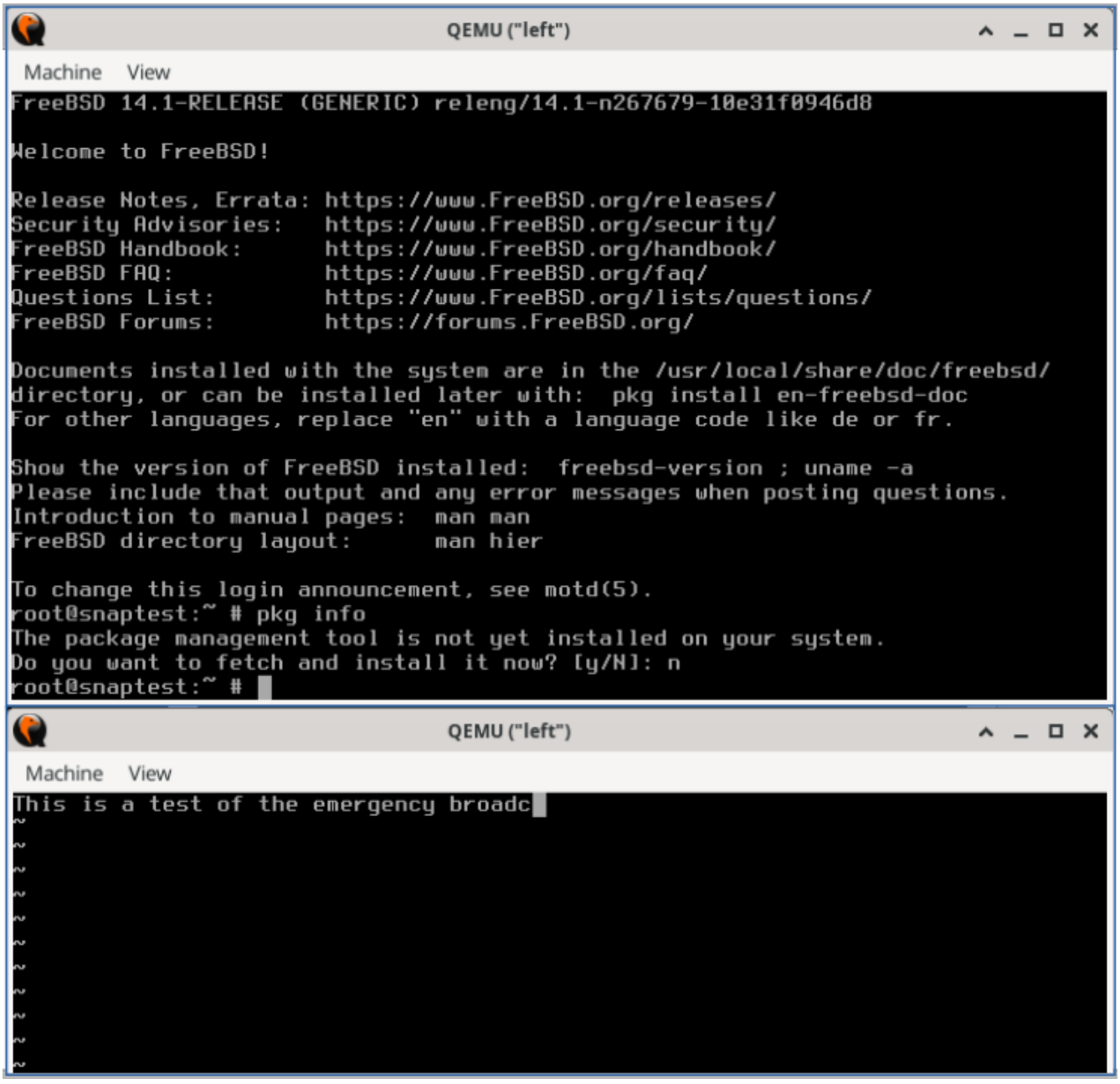



*Figure 72. QEMU VM Before First Snapshot*

To generate the snapshot, enter `savevm` in the monitor. Be sure to give it a tag (such as `original_install`).

```
QEMU 9.0.1 monitor - type 'help' for more information
(qemu)
```

```
(qemu) savevm original_install
```

Next, in the main console window, install a package, such as zip(1) which has no dependencies. Once that completes, renter the monitor and create another snapshot (`snap1_pkg+zip`).

Figure 14 shows the results of the above commands and the output of the `info shapshots` command.

```
[jpb@jpbsd:~/QEMU/SCRIPTS]$ sudo /bin/sh left.sh
QEMU 9.0.1 monitor - type 'help' for more information
(qemu) savevm original_install
(qemu)
(qemu) savevm snap1_pkg+zip
(qemu)
(qemu) info snapshots
List of snapshots present on all disks:
ID      TAG               VM_SIZE                DATE        VM_CLOCK     ICOUNT
--      original_install  2.4 GiB 2024-09-03 21:27:21  0000:39:06.533          --
--      snap1_pkg+zip    2.79 GiB 2024-09-03 21:33:27  0000:44:59.614          --
(qemu)
(qemu)
(qemu)
```

*Figure 73. QEMU Using Monitor Commands for Snapshots*

Reboot the system, and before FreeBSD starts up, switch to the monitor and enter `stop`. The VM will stop.

Enter `loadvm` with the tag used above (here `original_install`).

```
QEMU 9.0.1 monitor - type 'help' for more information
(qemu) stop
(qemu) loadvm original_install
(qemu) cont
```

Immediately, the VM screen will switch to the exact moment the `savevm` command was entered above. Note that the VM is still stopped.

Enter `cont` to start the VM, switch to the editing session on `ttyv1`, and type one letter on the keyboard. The editor, still in insert mode, should respond accordingly. Any other programs running at the time the snapshot was taken should be unaffected.

The above steps show how a snapshot can be taken, the system modified, and then "rolled back" by restoring the previous snapshot.

By default QEMU stores snapshot data in the same file as the image. View the list of snapshots with qemu-img(1) as shown below in Figure 15.

```
[jpb@jpbsd:~/QEMU/VM]$ qemu-img info left.qcow2
image: left.qcow2
file format: qcow2
virtual size: 16 GiB (17179869184 bytes)
disk size: 3.76 GiB
cluster_size: 65536
Snapshot list:
ID      TAG               VM_SIZE                DATE        VM_CLOCK     ICOUNT
1       original_install  2.4 GiB 2024-09-03 21:27:21  0000:39:06.533         --
2       snap1_pkg+zip    2.79 GiB 2024-09-03 21:33:27  0000:44:59.614         --
Format specific information:
    compat: 1.1
    compression type: zlib
    lazy refcounts: false
    refcount bits: 16
    corrupt: false
    extended l2: false
Child node '/file':
    filename: left.qcow2
    protocol type: file
    file length: 7.2 GiB (7735148544 bytes)
    disk size: 3.76 GiB
[jpb@jpbsd:~/QEMU/VM]$
```

*Figure 74. QEMU Using qemu-img(1) to Examine Snapshots*

### 24.6.7. Using QEMU USB Devices

QEMU supports the creation of virtual USB devices that are backed by an image file. These are virtual USB devices that can be partitioned, formatted, mounted, and used just like a real USB device.

```
/usr/local/bin/qemu-system-x86_64  -monitor stdio \
  -cpu qemu64 \
  -vga cirrus \
  -m 4096  -smp 4   \
  -cdrom ../ISO/fbsd.iso \
  -boot order=cd,menu=on \
  -drive if=none,id=usbstick,format=raw,file=../VM/foo.img \
  -usb \
  -device usb-ehci,id=ehci \
  -device usb-storage,bus=ehci.0,drive=usbstick \
  -device usb-mouse \
  -blockdev driver=file,node-name=img1,filename=../VM/right.qcow2 \
  -blockdev driver=qcow2,node-name=drive0,file=img1 \
  -device virtio-blk-pci,drive=drive0,bootindex=1  \
  -netdev tap,id=nd0,ifname=tap1,script=no,downscript=no,br=bridge0 \
  -device e1000,netdev=nd0,mac=02:72:69:67:68:74 \
  -name \"right\"
```

This configuration includes a `-drive` specification with the `id=usbstick`, raw format, and an image file (must be created by qemu-img(1)). The next line contains the `-device usb-ehci` specification for

a USB EHCI controller, with `id=ehci`. Finally, a `-device usb-storage` specification ties the above drive to the EHCI USB bus.

When the system is booted, FreeBSD will recognize a USB hub, add the attached USB device, and assign it to `da0` as shown in Figure 16.

```
.usb_msc_auto_quirk: UQ_MSC_NO_GETMAXLUN set for USB mass storage device QEMU QEMU USB HARDDRIVE (0x46f4:0x0001)
.ugen1.2: <QEMU QEMU USB HARDDRIVE> at usbus1
umass0 on uhub0
umass0: <QEMU QEMU USB HARDDRIVE, class 0/0, rev 2.00/0.00, addr 2> on usbus1
umass0:  SCSI over Bulk-Only; quirks = 0x0100
umass0:2:0: Attached to scbus2
da0 at umass-sim0 bus 0 scbus2 target 0 lun 0
da0: <QEMU QEMU HARDDISK 2.5+> Fixed Direct Access SPC-3 SCSI device
da0: Serial Number 1-0000:00:03.0-1
da0: 40.000MB/s transfers
da0: 15360MB (31457280 512 byte sectors)
da0: quirks=0x2<NO_6_BYTE>
```

*Figure 75. QEMU Created USB Hub and Mass Storage Device*

The device is ready to be partitioned with gpart(8), and formatted with newfs(8). Because the USB device is backed by a qemu-img(1) created file, data written to the device will persist across reboots.

### 24.6.8. Using Host USB Devices via Passthrough

QEMU USB passthrough support is listed as experimental in version 9.0.1 (Summer, 2024). However, the following steps show how a USB stick mounted on the host can be used by the guest VM.

For more information and examples, see:

- https://www.qemu.org/docs/master/system/devices/usb.html

The upper part of Figure 17 shows the QEMU monitor commands:

- `info usbhost` shows information about all USB devices on the host system. Find the desired USB device on the host system and note the two hexadecimal values on that line. (In the example below the host USB device is a Memorex Mini, with vendorid 0718, and productid 0619.) Use the two values shown by the `info usbhost` command in the `device_add` step below.
- `device_add` adds a USB device to the guest VM.

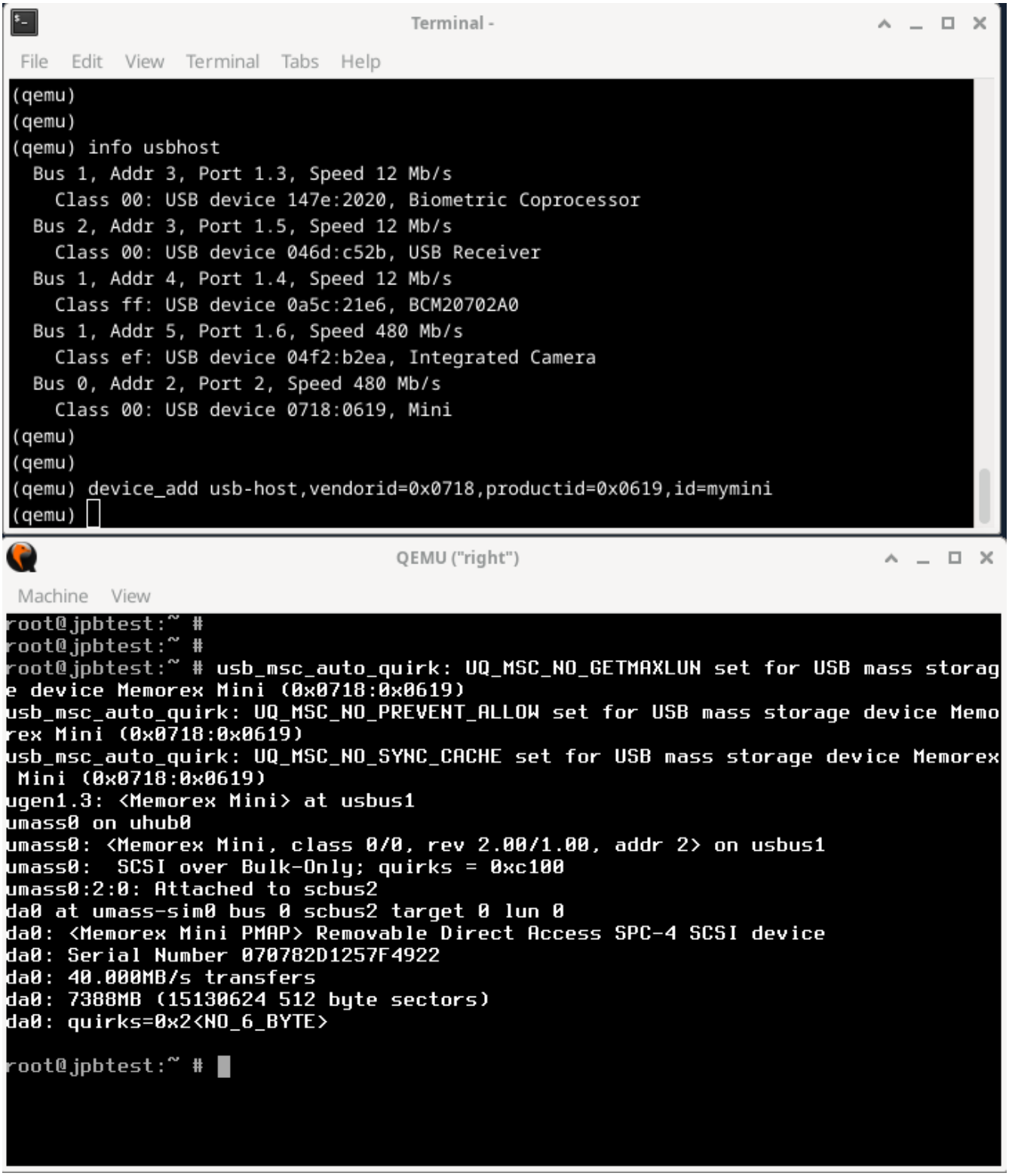



*Figure 76. QEMU Monitor Commands to Access a USB Device on the Host*

As before, once `device_add` completes, the FreeBSD kernel recognizes a new USB device, as shown in the lower half of the .

Using the new device is shown in Figure 18.

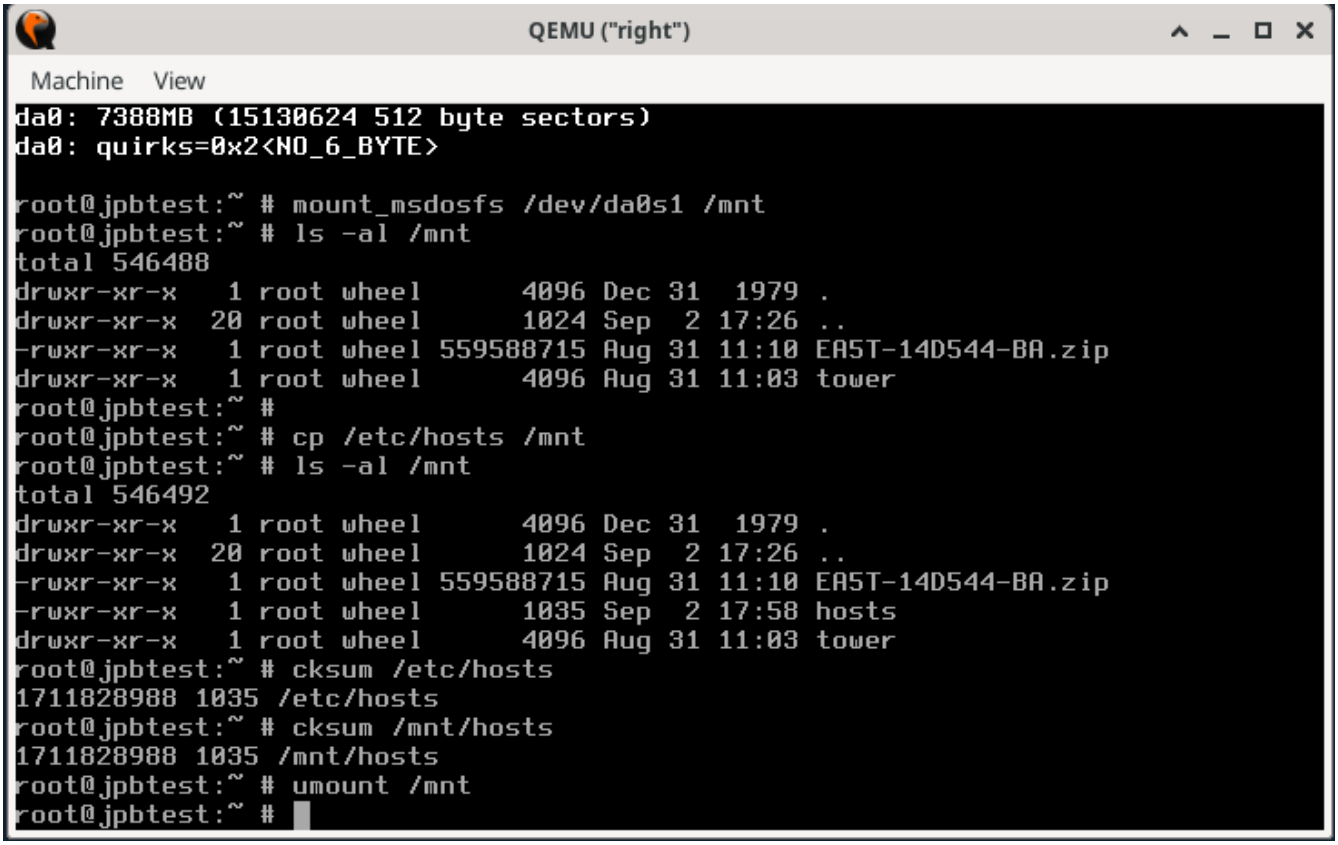



*Figure 77. Using the Host USB Device via Passthrough*

If the USB device is formatted as a FAT16 or FAT32 filesystem it can be mounted as an MS-DOS™ filesystem with mount_msdosfs(8) as in the example shown. The `/etc/hosts` file is copied to the newly mounted drive and checksums are taken to verify the integrity of the file on the USB device. The device is then unmounted with umount(8).

If the USB device is formatted with NTFS it is necessary to install the `fusefs-ntfs` package and use ntfs-3g(8) to access the device:

```
# pkg install fusefs-ntfs
# kldload fusefs
# gpart show da1
# ntfs-3g /dev/da1s1 /mnt

Access the drive as needed.  When finished:

# umount /mnt
```

Change the above device identifiers to match the installed hardware. Consult ntfs-3g(8) for additional information on working with NTFS filesystems.

### 24.6.9. QEMU on FreeBSD Summary

As noted above, QEMU works with several different hypervisor accelerators.

The list of Virtualization Accelerators supported by QEMU includes:

- KVM on Linux supporting 64 bit Arm, MIPS, PPC, RISC-V, s390x, and x86
- Xen on Linux as dom0 supporting Arm, x86
- Hypervisor Framework (hvf) on MacOS supporting x86 and Arm (both 64 bit only)
- Windows Hypervisor Platform (whpx) on Windows supporting x86
- NetBSD Virtual Machine Monitor (nvmm) on NetBSD supporting x86
- Tiny Code Generator (tcg) on Linux and other POSIX, Windows, MacOS supporting Arm, x86, Loongarch64, MIPS, PPC, s390x, and Sparc64.

All the examples in this section used the Tiny Code Generator (tcg) accelerator as that is the only supported accelerator on FreeBSD at present.

# 24.7. FreeBSD as a Host with bhyve

The bhyve BSD-licensed hypervisor became part of the base system with FreeBSD 10.0-RELEASE. This hypervisor supports several guests, including FreeBSD, OpenBSD, many Linux® distributions, and Microsoft Windows®. By default, bhyve provides access to a serial console and does not emulate a graphical console. Virtualization offload features of newer CPUs are used to avoid the legacy methods of translating instructions and manually managing memory mappings.

The bhyve design requires

- an Intel® processor that supports Intel Extended Page Tables (EPT),
- or an AMD® processor that supports AMD Rapid Virtualization Indexing (RVI), or Nested Page Tables (NPT),
- or an ARM® aarch64 CPU.

Only pure ARMv8.0 virtualization is supported on ARM, the Virtualization Host Extensions are not currently used. Hosting Linux® guests or FreeBSD guests with more than one vCPU requires VMX unrestricted mode support (UG).

The easiest way to tell if an Intel or AMD processor supports bhyve is to run dmesg or look in /var/run/dmesg.boot for the POPCNT processor feature flag on the Features2 line for AMD® processors or EPT and UG on the VT-x line for Intel® processors.

## 24.7.1. Preparing the Host

The first step to creating a virtual machine in bhyve is configuring the host system. First, load the bhyve kernel module:

```
# kldload vmm
```

There are several ways to connect a virtual machine guest to a host's network; one straightforward way to accomplish this is to create a tap interface for the network device in the virtual machine to attach to. For the network device to participate in the network, also create a bridge interface containing the tap interface and the physical interface as members. In this example, the physical

interface is *igb0*:

```
# ifconfig tap0 create
# sysctl net.link.tap.up_on_open=1
net.link.tap.up_on_open: 0 -> 1
# ifconfig bridge0 create
# ifconfig bridge0 addm igb0 addm tap0
# ifconfig bridge0 up
```

### 24.7.2. Creating a FreeBSD Guest

Create a file to use as the virtual disk for the guest machine. Specify the size and name of the virtual disk:

```
# truncate -s 16G guest.img
```

Download an installation image of FreeBSD to install:

```
# fetch https://download.freebsd.org/releases/ISO-IMAGES/14.0/FreeBSD-14.0-RELEASE-
amd64-bootonly.iso
FreeBSD-14.0-RELEASE-amd64-bootonly.iso                426 MB   16 MBps    22s
```

FreeBSD comes with an example script `vmrun.sh` for running a virtual machine in bhyve. It will start the virtual machine and run it in a loop, so it will automatically restart if it crashes. `vmrun.sh` takes several options to control the configuration of the machine, including:

- `-c` controls the number of virtual CPUs,
- `-m` limits the amount of memory available to the guest,
- `-t` defines which tap device to use,
- `-d` indicates which disk image to use,
- `-i` tells bhyve to boot from the CD image instead of the disk, and
- `-I` defines which CD image to use.

The last parameter is the name of the virtual machine and is used to track the running machines. The following command lists all available program argument options:

```
# sh /usr/share/examples/bhyve/vmrun.sh -h
```

This example starts the virtual machine in installation mode:

```
# sh /usr/share/examples/bhyve/vmrun.sh -c 1 -m 1024M -t tap0 -d guest.img \
    -i -I FreeBSD-14.0-RELEASE-amd64-bootonly.iso guestname
```

The virtual machine will boot and start the installer. After installing a system in the virtual machine, when the system asks about dropping into a shell at the end of the installation, choose **[ Yes ]**.

Reboot the virtual machine. While rebooting the virtual machine causes bhyve to exit, the vmrun.sh script runs `bhyve` in a loop and will automatically restart it. When this happens, choose the reboot option from the boot loader menu to escape the loop. Now the guest can be started from the virtual disk:

```
# sh /usr/share/examples/bhyve/vmrun.sh -c 4 -m 1024M -t tap0 -d guest.img guestname
```

### 24.7.3. Creating a Linux® Guest

Linux guests can be booted either like any other regular UEFI-based guest virtual machine, or alternatively, use the sysutils/grub2-bhyve port.

To do this, first ensure that the port is installed, then create a file to use as the virtual disk for the guest machine:

```
# truncate -s 16G linux.img
```

Starting a Linux virtual machine with `grub2-bhyve` is a two-step process.

1. First a kernel must be loaded, then the guest can be started.
2. The Linux® kernel is loaded with sysutils/grub2-bhyve.

Create a device.map that grub will use to map the virtual devices to the files on the host system:

```
(hd0) ./linux.img
(cd0) ./somelinux.iso
```

Use sysutils/grub2-bhyve to load the Linux® kernel from the ISO image:

```
# grub-bhyve -m device.map -r cd0 -M 1024M linuxguest
```

This will start grub. If the installation CD contains a grub.cfg, a menu will be displayed. If not, the `vmlinuz` and `initrd` files must be located and loaded manually:

```
grub> ls
(hd0) (cd0) (cd0,msdos1) (host)
grub> ls (cd0)/isolinux
boot.cat boot.msg grub.conf initrd.img isolinux.bin isolinux.cfg memtest
splash.jpg TRANS.TBL vesamenu.c32 vmlinuz
grub> linux (cd0)/isolinux/vmlinuz
grub> initrd (cd0)/isolinux/initrd.img
```

```
grub> boot
```

Now that the Linux® kernel is loaded, the guest can be started:

```
# bhyve -A -H -P -s 0:0,hostbridge -s 1:0,lpc -s 2:0,virtio-net,tap0 \
    -s 3:0,virtio-blk,./linux.img -s 4:0,ahci-cd,./somelinux.iso \
    -l com1,stdio -c 4 -m 1024M linuxguest
```

The system will boot and start the installer. After installing a system in the virtual machine, reboot the virtual machine. This will cause bhyve to exit. The instance of the virtual machine needs to be destroyed before it can be started again:

```
# bhyvectl --destroy --vm=linuxguest
```

Now the guest can be started directly from the virtual disk. Load the kernel:

```
# grub-bhyve -m device.map -r hd0,msdos1 -M 1024M linuxguest
grub> ls
(hd0) (hd0,msdos2) (hd0,msdos1) (cd0) (cd0,msdos1) (host)
(lvm/VolGroup-lv_swap) (lvm/VolGroup-lv_root)
grub> ls (hd0,msdos1)/
lost+found/ grub/ efi/ System.map-2.6.32-431.el6.x86_64 config-2.6.32-431.el6.x
86_64 symvers-2.6.32-431.el6.x86_64.gz vmlinuz-2.6.32-431.el6.x86_64
initramfs-2.6.32-431.el6.x86_64.img
grub> linux (hd0,msdos1)/vmlinuz-2.6.32-431.el6.x86_64 root=/dev/mapper/VolGroup-
lv_root
grub> initrd (hd0,msdos1)/initramfs-2.6.32-431.el6.x86_64.img
grub> boot
```

Boot the virtual machine:

```
# bhyve -A -H -P -s 0:0,hostbridge -s 1:0,lpc -s 2:0,virtio-net,tap0 \
    -s 3:0,virtio-blk,./linux.img -l com1,stdio -c 4 -m 1024M linuxguest
```

Linux® will now boot in the virtual machine and eventually presents the login prompt. Login and use the virtual machine. When finished, reboot the virtual machine to exit bhyve. Destroy the virtual machine instance:

```
# bhyvectl --destroy --vm=linuxguest
```

### 24.7.4. Booting bhyve Virtual Machines with UEFI Firmware

In addition to `bhyveload` and `grub-bhyve`, the bhyve hypervisor can also boot virtual machines using the UEFI firmware. This option may support guest operating systems that are not supported by the

other loaders.

To make use of the UEFI support in bhyve, first obtain the UEFI firmware images. This can be done by installing sysutils/bhyve-firmware port or package.

With the firmware in place, add the flags `-l bootrom,/path/to/firmware` to the bhyve command line. The actual bhyve command may look like this:

```
# bhyve -AHP -s 0:0,hostbridge -s 1:0,lpc \
    -s 2:0,virtio-net,tap1 -s 3:0,virtio-blk,./disk.img \
    -s 4:0,ahci-cd,./install.iso -c 4 -m 1024M \
    -l bootrom,/usr/local/share/uefi-firmware/BHYVE_UEFI.fd \
    guest
```

To allow a guest to store UEFI variables, use a variables file appended to the `-l` flag. Note that bhyve will write guest modifications to the given variables file. Therefore, be sure to first create a per-guest-copy of the variables template file:

```
# cp /usr/local/share/uefi-firmware/BHYVE_UEFI_VARS.fd /path/to/vm-
image/BHYVE_UEFI_VARS.fd
```

Then, add that variables file to the bhyve arguments:

```
# bhyve -AHP -s 0:0,hostbridge -s 1:0,lpc \
    -s 2:0,virtio-net,tap1 -s 3:0,virtio-blk,./disk.img \
    -s 4:0,ahci-cd,./install.iso -c 4 -m 1024M \
    -l bootrom,/usr/local/share/uefi-firmware/BHYVE_UEFI.fd,/path/to/vm-
image/BHYVE_UEFI_VARS.fd \
    guest
```



Some Linux distributions require the use of UEFI variables to store the path for their UEFI boot file (using `linux64.efi` or `grubx64.efi` instead of `bootx64.efi`, for example). It is therefore recommended to use a variables file for Linux virtual machines to avoid having to manually alter the boot partition files.

To view or modify the variables file contents, use efivar(8) from the host.

sysutils/bhyve-firmware also contains a CSM-enabled firmware, to boot guests with no UEFI support in legacy BIOS mode:

```
# bhyve -AHP -s 0:0,hostbridge -s 1:0,lpc \
    -s 2:0,virtio-net,tap1 -s 3:0,virtio-blk,./disk.img \
    -s 4:0,ahci-cd,./install.iso -c 4 -m 1024M \
    -l bootrom,/usr/local/share/uefi-firmware/BHYVE_UEFI_CSM.fd \
    guest
```

### 24.7.5. Graphical UEFI Framebuffer for bhyve Guests

The UEFI firmware support is particularly useful with predominantly graphical guest operating systems such as Microsoft Windows®.

Support for the UEFI-GOP framebuffer may also be enabled with the `-s 29,fbuf,tcp=0.0.0.0:5900` flags. The framebuffer resolution may be configured with `w=800` and `h=600`, and bhyve can be instructed to wait for a VNC connection before booting the guest by adding `wait`. The framebuffer may be accessed from the host or over the network via the VNC protocol. Additionally, `-s 30,xhci,tablet` can be added to achieve precise mouse cursor synchronization with the host.

The resulting bhyve command would look like this:

```
# bhyve -AHP -s 0:0,hostbridge -s 31:0,lpc \
    -s 2:0,virtio-net,tap1 -s 3:0,virtio-blk,./disk.img \
    -s 4:0,ahci-cd,./install.iso -c 4 -m 1024M \
    -s 29,fbuf,tcp=0.0.0.0:5900,w=800,h=600,wait \
    -s 30,xhci,tablet \
    -l bootrom,/usr/local/share/uefi-firmware/BHYVE_UEFI.fd \
    guest
```

Note, in BIOS emulation mode, the framebuffer will cease receiving updates once control is passed from firmware to guest operating system.

### 24.7.6. Creating a Microsoft Windows® Guest

Setting up a guest for Windows versions 10 or earlier can be done directly from the original installation media and is a relatively straightforward process. Aside from minimum resource requirements, running Windows as guest requires

- wiring virtual machine memory (flag `-w`) and
- booting with an UEFI bootrom.

An example for booting a virtual machine guest with a Windows installation ISO:

```
bhyve \
      -c 2 \
      -s 0,hostbridge \
      -s 3,nvme,windows2016.img \
      -s 4,ahci-cd,install.iso \
      -s 10,virtio-net,tap0 \
      -s 31,lpc \
      -s 30,xhci,tablet \
      -l bootrom,/usr/local/share/uefi-firmware/BHYVE_UEFI.fd \
      -m 8G -H -w \
      windows2016
```

Only one or two VCPUs should be used during installation but this number can be increased once

Windows is installed.

VirtIO drivers must be installed to use the defined `virtio-net` network interface. An alternative is to switch to E1000 (Intel E82545) emulation by changing `virtio-net` to `e1000` in the above command line. However, performance will be impacted.

#### 24.7.6.1. Creating a Windows 11 Guest

Beginning with Windows 11, Microsoft introduced a hardware requirement for a TPM 2 module. bhyve supports passing a hardware TPM through to a guest. The installation media can be modified to disable the relevant hardware checks. A detailed description for this process can be found on the FreeBSD Wiki.



Modifying Windows installation media and running Windows guests without a TPM module are unsupported by the manufacturer. Consider the application and use case before implementing such approaches.

### 24.7.7. Using ZFS with bhyve Guests

If ZFS is available on the host machine, using ZFS volumes instead of disk image files can provide significant performance benefits for the guest VMs. A ZFS volume can be created by:

```
# zfs create -V16G -o volmode=dev zroot/linuxdisk0
```

When starting the VM, specify the ZFS volume as the disk drive:

```
# bhyve -A -H -P -s 0:0,hostbridge -s 1:0,lpc -s 2:0,virtio-net,tap0 \
    -s3:0,virtio-blk,/dev/zvol/zroot/linuxdisk0 \
    -l com1,stdio -c 4 -m 1024M linuxguest
```

When using ZFS for the host as well as inside a guest, keep in mind the competing memory pressure of both systems caching the virtual machine's contents. To alleviate this, consider setting the host's ZFS filesystems to use metadata-only cache. To do this, apply the following settings to ZFS filesystems on the host, replacing `<name>` with the name of the specific zvol dataset name of the virtual machine.

```
# zfs set primarycache=metadata <name>
```

### 24.7.8. Creating a Virtual Machine Snapshot

Modern hypervisors allow their users to create "snapshots" of their state; such a snapshot includes a guest's disk, CPU, and memory contents. A snapshot can usually be taken independent of whether the guest is running or shut down. One can then reset and return the virtual machine to the precise state when the snapshot was taken.

#### 24.7.8.1. ZFS Snapshots

Using ZFS volumes as the backing storage for a virtual machine enables the snapshotting of the guest's disk. For example:

```
zfs snapshot zroot/path/to/zvol@snapshot_name
```

Though it is possible to snapshot a ZFS volume this way while the guest is running, keep in mind that the contents of the virtual disk may be in an inconsistent state while the guest is active. It is therefore recommended to first shutdown or pause the guest before executing this command. Pausing a guest is not supported by default and needs to be enabled first (see Memory and CPU Snapshots)



Rolling back a ZFS zvol to a snapshot while a virtual machine is using it may corrupt the file system contents and crash the guest. All unsaved data in the guest will be lost and modifications since the last snapshot may get destroyed.

A second rollback may be required once the virtual machine is shut down to restore the file system to a useable state. This in turn will ultimately destroy any changes made after the snapshot.

#### 24.7.8.2. Memory and CPU Snapshots (Experimental Feature)

As of FreeBSD 13, bhyve has an experimental "snapshot" feature for dumping a guest's memory and CPU state to a file and then halting the virtual machine. The guest can be resumed from the snapshot file contents later.

However, this feature is not enabled by default and requires the system to be rebuilt from source. See Building from Source for an in-depth description on the process of compiling the kernel with custom options.



The functionality is not ready for production use and limited to work for specific virtual machine configurations. There are multiple limitations:

- `nvme` and `virtio-blk` storage backends do not work yet
- snapshots are only supported when the guest uses a single kind of each device, i.e. if there is more than one `ahci-hd` disk attached, snapshot creation will fail
- additionally, the feature may be reasonably stable on Intel, but it probably won't work on AMD CPUs.



Make sure the /usr/src directory is up-to date before taking the following steps. See Updating the Source for the detailed procedure how to do this.

First, add the following to /etc/src.conf:

```
WITH_BHYVE_SNAPHOT=yes
BHYVE_SNAPSHOT=1
```

```
MK_BHYVE_SNAPSHOT=yes
```



If the system was partially or wholly rebuilt, it is recommended to run

```
# cd /usr/src
# make cleanworld
```

before proceeding.

Then follow the steps outlined in the Quick Start section of the Updating FreeBSD from Source chapter to build and install world and kernel.

To verify successful activation of the snapshot feature, enter

```
# bhyvectl --usage
```

and check if the output lists a `--suspend` flag. If the flag is missing, the feature did not activate correctly.

Then, snapshot and suspend a running virtual machine of choice:

```
# bhyvectl --vm=vmname --suspend=/path/to/snapshot/filename
```



Provide an absolute path and filename to `--suspend`. Otherwise, bhyve will write snapshot data to whichever directory bhyve was started from.

Make sure to write the snapshot data to a secure directory. The generated output contains a full memory dump of the guest and may thus contain sensitive data (i.e. passwords)!

This creates three files:

- memory snapshot - named like the input to `--suspend`
- kernel file - name like the input to `--suspend` with the suffix .kern
- metadata - contains meta data about the system state, named with the suffix .meta

To restore a guest from a snapshot, use the `-r` flag with `bhyve`:

```
# bhyve -r /path/to/snapshot/filename
```

Restoring a guest snapshot on a different CPU architecture will not work. Generally, attempting to restore on a system not identical to the snapshot creator will likely fail.

### 24.7.9. Jailing bhyve

For improved security and separation of virtual machines from the host operating system, it is possible to run bhyve in a jail. See Jails for an in-depth description of jails and their security benefits.

#### 24.7.9.1. Creating a Jail for bhyve

First, create a jail environment. If using a UFS file system, simply run:

```
# mkdir -p /jails/bhyve
```

If using a ZFS filesystem, use the following commands:

```
# zfs create zroot/jails
# zfs create zroot/jails/bhyve
```

Then create a ZFS zvol for the virtual machine `bhyvevm0`:

```
# zfs create zroot/vms
# zfs create -V 20G zroot/vms/bhyvevm0
```

If not using ZFS, use the following commands to create a disk image file directly in the jail directory structure:

```
# mkdir /jails/bhyve/vms
# truncate -s 20G /jails/bhyve/vms/bhyvevm0
```

Download a FreeBSD image, preferably a version equal to or older than the host and extract it into the jail directory:

```
# cd /jails
# fetch -o base.txz http://ftp.freebsd.org/pub/FreeBSD/releases/amd64/13.2-
RELEASE/base.txz
# tar -C /jails/bhyve -xvf base.txz
```



Running a higher FreeBSD version in a jail than the host is unsupported (i.e. running 14.0-RELEASE in a jail, embedded in a 13.2-RELEASE host).

Next, add a devfs ruleset to /etc/devfs.rules:

```
[devfsrules_jail_bhyve=100]
add include $devfsrules_hide_all
add include $devfsrules_unhide_login
```

```
add path 'urandom' unhide
add path 'random' unhide
add path 'crypto' unhide
add path 'shm' unhide
add path 'zero' unhide
add path 'null' unhide
add path 'mem' unhide
add path 'vmm' unhide
add path 'vmm/*' unhide
add path 'vmm.io' unhide
add path 'vmm.io/*' unhide
add path 'nmdmbhyve*' unhide
add path 'zvol' unhide
add path 'zvol/zroot' unhide
add path 'zvol/zroot/vms' unhide
add path 'zvol/zroot/vms/bhyvevm0' unhide
add path 'zvol/zroot/vms/bhyvevm1' unhide
add path 'tap10*' unhide
```



If there's another devfs rule with the numeric ID 100 in the /etc/devfs.rules file, replace the one in the listing with another yet unused ID number.



If not using a ZFS filesystem, skip the related zvol rules in /etc/devfs.rules:

```
add path 'zvol' unhide
add path 'zvol/zroot' unhide
add path 'zvol/zroot/vms' unhide
add path 'zvol/zroot/vms/bhyvevm0' unhide
add path 'zvol/zroot/vms/bhyvevm1' unhide
```

These rules will cause bhyve to

- create a virtual machine with disk volumes called `bhyvevm0` and `bhyvevm1`,
- use tap network interfaces with the name prefix `tap10`. That means, valid interface names will be `tap10`, `tap100`, `tap101`, … `tap109`, `tap1000` and so on.

  Limiting the access to a subset of possible tap interface names will prevent the jail (and thus bhyve) from seeing tap interfaces of the host and other jails.

- use nmdm devices prefixed with "bhyve", i.e. /dev/nmdmbhyve0.

Those rules can be expanded and varied with different guest and interface names as desired.



When intending to use bhyve on the host as well as in a one or more jails, remember that tap and nmdm interface names will operate in a shared environment. For example, use /dev/nmdmbhyve0 only either for bhyve on the host or in a jail.

Restart devfs for the changes to be loaded:

```
# service devfs restart
```

Then add a definition for the new jail into /etc/jail.conf or /etc/jail.conf.d. Replace the interface number $if and IP address with personal variations.

*Example 33. Using NAT or routed traffic with a firewall*

```
bhyve {
        $if = 0;
        exec.prestart = "/sbin/ifconfig epair${if} create up";
        exec.prestart += "/sbin/ifconfig epair${if}a up";
        exec.prestart += "/sbin/ifconfig epair${if}a name ${name}0";
        exec.prestart += "/sbin/ifconfig epair${if}b name jail${if}";
        exec.prestart += "/sbin/ifconfig ${name}0 inet 192.168.168.1/27";
        exec.prestart += "/sbin/sysctl net.inet.ip.forwarding=1";

        exec.clean;

        host.hostname = "the-hostname-here";
        vnet;
        vnet.interface = "jail${if}";
        path = "/jails/${name}";
        persist;
        securelevel = 3;
        devfs_ruleset = 100;
        mount.devfs;

        allow.vmm;

        exec.start += "/bin/sh /etc/rc";
        exec.stop = "/bin/sh /etc/rc.shutdown";

        exec.poststop += "/sbin/ifconfig ${name}0 destroy";
}
```

This example assumes use of a firewall like `pf` or `ipfw` to NAT the jail traffic. See the Firewalls chapter for more details on the available options to implement this.

*Example 34. Using a bridged network connection*

```
bhyve {
        $if = 0;
        exec.prestart = "/sbin/ifconfig epair${if} create up";
        exec.prestart += "/sbin/ifconfig epair${if}a up";
        exec.prestart += "/sbin/ifconfig epair${if}a name ${name}0";
```

```
        exec.prestart += "/sbin/ifconfig epair${if}b name jail${if}";
        exec.prestart += "/sbin/ifconfig bridge0 addm ${name}0";
        exec.prestart += "/sbin/sysctl net.inet.ip.forwarding=1";

        exec.clean;

        host.hostname = "the-hostname-here";
        vnet;
        vnet.interface = "jail${if}";
        path = "/jails/${name}";
        persist;
        securelevel = 3;
        devfs_ruleset = 100;
        mount.devfs;

        allow.vmm;

        exec.start += "/bin/sh /etc/rc";
        exec.stop = "/bin/sh /etc/rc.shutdown";

        exec.poststop += "/sbin/ifconfig ${name}0 destroy";
}
```



Having previously replaced the devfs ruleset ID 100 in /etc/devfs.rules with a custom unique number, remember to replace the numeric ID also in the jails.conf too.

#### 24.7.9.2. Configuring the Jail

To start the jail for the first time and do some additional configuration work, enter:

```
# cp /etc/resolv.conf /jails/bhyve/etc
# service jail onestart bhyve
# jexec bhyve
# sysrc ifconfig_jail0="inet 192.168.168.2/27"
# sysrc defaultrouter="192.168.168.1"
# sysrc sendmail_enable=NONE
# sysrc cloned_interfaces="tap100"
# exit
```

Restart and enable the jail:

```
# sysrc jail_enable=YES
# service jail restart bhyve
```

Afterwards, create a virtual machine within the jail. For a FreeBSD guest, download an installation

ISO first:

```
# jexec bhyve
# cd /vms
# fetch -o freebsd.iso https://download.freebsd.org/releases/ISO-IMAGES/14.0/FreeBSD-14.0-RELEASE-amd64-bootonly.iso
```

#### 24.7.9.3. Creating a Virtual Machine Inside the Jail

To create a virtual machine, use `bhyvectl` to initialize it first:

```
# jexec bhyve
# bhyvectl --create --vm=bhyvevm0
```



Creating the guest with `bhyvectl` may be required when initiating the virtual machine from a jail. Skipping this step may cause the following error message when starting `bhyve`:

```
vm_open: vm-name could not be opened. No such file or directory
```

Finally, use the preferred way of starting the guest.

*Example 35. Starting with `vmrun.sh` and ZFS*

Using `vmrun.sh` on a ZFS filesystems:

```
# jexec bhyve
# sh /usr/share/examples/bhyve/vmrun.sh -c 1 -m 1024M \
     -t tap100 -d /dev/zvol/zroot/vms/bhyvevm0 -i -I /vms/FreeBSD-14.0-RELEASE-amd64-bootonly.iso bhyvevm0
```

*Example 36. Starting with `vmrun.sh` and UFS*

Using `vmrun.sh` on a UFS filesystem:

```
# jexec bhyve
# sh /usr/share/examples/bhyve/vmrun.sh -c 1 -m 1024M \
     -t tap100 -d /vms/bhyvevm0 -i -I /vms/FreeBSD-14.0-RELEASE-amd64-bootonly.iso
bhyvevm0
```

*Example 37. Starting bhyve for an UEFI guest with ZFS*

When wanting to use an UEFI guest, remember to first install the required firmware package sysutils/bhyve-firmware in the jail:

```
# pkg -j bhyve install bhyve-firmware
```

Then use bhyve directly:

```
# bhyve -A -c 4 -D -H -m 2G \
        -s 0,hostbridge \
        -s 1,lpc \
        -s 2,virtio-net,tap100 \
        -s 3,virtio-blk,/dev/zvol/zroot/vms/bhyvevm0 \
    -s 4,ahci-cd,/vms/FreeBSD-14.0-RELEASE-amd64-bootonly.iso \
        -s 31,fbuf,tcp=127.0.0.1:5900,w=1024,h=800,tablet \
        -l bootrom,/usr/local/share/uefi-firmware/BHYVE_UEFI.fd \
        -l com1,/dev/nmdbbhyve0A \
        bhyvevm0
```

This allows connecting to the virtual machine bhyvevm0 through VNC as well as a serial console at /dev/nmdbbhyve0B.

### 24.7.10. Virtual Machine Consoles

It is advantageous to wrap the bhyve console in a session management tool such as sysutils/tmux or sysutils/screen in order to detach and reattach to the console. It is also possible to have the console of bhyve be a null modem device that can be accessed with cu. To do this, load the nmdm kernel module and replace `-l com1,stdio` with `-l com1,/dev/nmdm0A`. The /dev/nmdm devices are created automatically as needed, where each is a pair, corresponding to the two ends of the null modem cable (/dev/nmdm0A and /dev/nmdm0B). See nmdm(4) for more information.

```
# kldload nmdm
# bhyve -A -H -P -s 0:0,hostbridge -s 1:0,lpc -s 2:0,virtio-net,tap0 -s 3:0,virtio-
blk,./linux.img \
    -l com1,/dev/nmdm0A -c 4 -m 1024M linuxguest
# cu -l /dev/nmdm0B
Connected

Ubuntu 13.10 handbook ttyS0

handbook login:
```

To disconnect from a console, enter a newline (i.e. press RETURN) followed by tilde (~), and finally dot (.). Keep in mind that only the connection is dropped while the login session remains active. Another user connecting to the same console could therefore make use of any active sessions without having to first authenticate. For security reasons, it's therefore recommended to logout before disconnecting.

The number in the nmdm device path must be unique for each virtual machine and must not be used by any other processes before bhyve starts. The number can be chosen arbitrarily and does

not need to be taken from a consecutive sequence of numbers. The device node pair (i.e. /dev/nmdm0a and /dev/nmdm0b) are created dynamically when bhyve connects its console and destroyed when it shuts down. Keep this in mind when creating scripts to start the virtual machines: make sure that all virtual machines are assigned unique nmdm devices.

### 24.7.11. Managing Virtual Machines

A device node is created in /dev/vmm for each virtual machine. This allows the administrator to easily see a list of the running virtual machines:

```
# ls -al /dev/vmm
total 1
dr-xr-xr-x   2 root  wheel    512 Mar 17 12:19 ./
dr-xr-xr-x  14 root  wheel    512 Mar 17 06:38 ../
crw-------   1 root  wheel  0x1a2 Mar 17 12:20 guestname
crw-------   1 root  wheel  0x19f Mar 17 12:19 linuxguest
crw-------   1 root  wheel  0x1a1 Mar 17 12:19 otherguest
```

A specified virtual machine can be destroyed using `bhyvectl`:

```
# bhyvectl --destroy --vm=guestname
```

Destroying a virtual machine this way means killing it immediately. Any unsaved data will be lost, open files and filesystems may get corrupted. To gracefully shut down a virtual machine, send a `TERM` signal to its bhyve process instead. This triggers an ACPI shutdown event for the guest:

```
# ps ax | grep bhyve
17424  -  SC      56:48.27 bhyve: guestvm (bhyve)
# kill 17424
```

### 24.7.12. Tools and Utilities

There are numerous utilities and applications available in ports to help simplify setting up and managing bhyve virtual machines:

*Table 34. bhyve Managers*

| Name | License | Package | Documentation |
|---|---|---|---|
| vm-bhyve | BSD-2 | sysutils/vm-bhyve | Documentation |
| CBSD | BSD-2 | sysutils/cbsd | Documentation |
| Virt-Manager | LGPL-3 | deskutils/virt-manager | Documentation |
| Bhyve RC Script | Unknown | sysutils/bhyve-rc | Documentation |
| bmd | BSD-2 | sysutils/bmd | Documentation |
| vmstated | BSD-2 | sysutils/vmstated | Documentation |

### 24.7.13. Persistent Configuration

In order to configure the system to start bhyve guests at boot time, some configuration file changes are required.

1. /etc/sysctl.conf

   When using tap interfaces as network backend, either manually set each used tap interface to UP or simply set the following sysctl:

   ```
   net.link.tap.up_on_open=1
   ```

2. /etc/rc.conf

   To connect the virtual machine's tap device to the network via a bridge, persisting the device settings in /etc/rc.conf is needed. Additionally, load the necessary kernel modules `vmm` for bhyve and `nmdm` for nmdm devices through the `kld_list` configuration variable. When configuring `ifconfig_bridge0`, make sure to replace `<ipaddr>/<netmask>` with the actual IP address of the physical interface (igb0 in this example) and remove IP settings from the physical device.

   ```
   # sysrc cloned_interfaces+="bridge0 tap0"
   # sysrc ifconfig_bridge0="inet <ipaddr>/<netmask> addm igb0 addm tap0"
   # sysrc kld_list+="nmdm vmm"
   # sysrc ifconfig_igb0="up"
   ```

*Example 38. Setting the IP for a bridge device*

For a host with an *igb0* interface connected to the network with IP `10.10.10.1` and netmask `255.255.255.0`, use the following commands:

```
# sysrc ifconfig_igb0="up"
# sysrc ifconfig_bridge0="inet 10.10.10.1/24 addm igb0 addm tap0"
# sysrc kld_list+="nmdm vmm"
# sysrc cloned_interfaces+="bridge0 tap0"
```



Modifying the IP address configuration of a system may terminate the current remote connection (e.g., via SSH), causing a lock out. Take precautions to maintain system access or make those modifications while logged in on a local terminal session.

## 24.8. FreeBSD as a Xen™-Host

Xen is a GPLv2-licensed type 1 hypervisor for Intel® and ARM® architectures. FreeBSD has included i386™ and AMD® 64-Bit DomU and Amazon EC2 unprivileged domain (virtual machine) support since FreeBSD 8.0 and includes Dom0 control domain (host) support in FreeBSD 11.0.

Support for para-virtualized (PV) domains has been removed from FreeBSD 11 in favor of hardware virtualized (HVM) domains, which provides better performance.

Xen™ is a bare-metal hypervisor, which means that it is the first program loaded after the BIOS. A special privileged guest called the Domain-0 (`Dom0` for short) is then started. The Dom0 uses its special privileges to directly access the underlying physical hardware, making it a high-performance solution. It is able to access the disk controllers and network adapters directly. The Xen™ management tools to manage and control the Xen™ hypervisor are also used by the Dom0 to create, list, and destroy VMs. Dom0 provides virtual disks and networking for unprivileged domains, often called `DomU`. Xen™ Dom0 can be compared to the service console of other hypervisor solutions, while the DomU is where individual guest VMs are run.

Xen™ can migrate VMs between different Xen™ servers. When the two xen hosts share the same underlying storage, the migration can be done without having to shut the VM down first. Instead, the migration is performed live while the DomU is running and there is no need to restart it or plan a downtime. This is useful in maintenance scenarios or upgrade windows to ensure that the services provided by the DomU are still provided. Many more features of Xen™ are listed on the Xen Wiki Overview page. Note that not all features are supported on FreeBSD yet.

### 24.8.1. Hardware Requirements for Xen™ Dom0

To run the Xen™ hypervisor on a host, certain hardware functionality is required. Running FreeBSD as a Xen host (Dom0) require Intel Extended Page Tables (EPT) or AMD Nested Page Tables (NPT) and Input/Output Memory Management Unit (IOMMU) support in the host processor.



In order to run a FreeBSD 13 Xen™ Dom0 the box must be booted using legacy boot (BIOS). FreeBSD 14 and newer supports booting as a Xen™ Dom0 in both BIOS and UEFI modes.

### 24.8.2. Xen™ Dom0 Control Domain Setup

Users should install the emulators/xen-kernel and sysutils/xen-tools packages, based on Xen™ 4.18.

Configuration files must be edited to prepare the host for the Dom0 integration after the Xen packages are installed. An entry to /etc/sysctl.conf disables the limit on how many pages of memory are allowed to be wired. Otherwise, DomU VMs with higher memory requirements will not run.

```
# echo 'vm.max_wired=-1' >> /etc/sysctl.conf
```

Another memory-related setting involves changing /etc/login.conf, setting the `memorylocked` option to `unlimited`. Otherwise, creating DomU domains may fail with `Cannot allocate memory` errors. After making the change to /etc/login.conf, run `cap_mkdb` to update the capability database. See Resource Limits for details.

```
# sed -i '' -e 's/memorylocked=64K/memorylocked=unlimited/' /etc/login.conf
# cap_mkdb /etc/login.conf
```

Add an entry for the Xen™ console to /etc/ttys:

```
# echo 'xc0     "/usr/libexec/getty Pc"         xterm   onifconsole  secure' >>
/etc/ttys
```

Selecting a Xen™ kernel in /boot/loader.conf activates the Dom0. Xen™ also requires resources like CPU and memory from the host machine for itself and other DomU domains. How much CPU and memory depends on the individual requirements and hardware capabilities. In this example, 8 GB of memory and 4 virtual CPUs are made available for the Dom0. The serial console is also activated, and logging options are defined.

The following command is used for Xen 4.7 packages:

```
# echo 'hw.pci.mcfg=0' >> /boot/loader.conf
# echo 'if_tap_load="YES"' >> /boot/loader.conf
# echo 'xen_kernel="/boot/xen"' >> /boot/loader.conf
# echo 'xen_cmdline="dom0_mem=8192M dom0_max_vcpus=4 dom0pvh=1 console=com1,vga
com1=115200,8n1 guest_loglvl=all loglvl=all"' >> /boot/loader.conf
```

For Xen versions 4.11 and higher, the following command should be used instead:

```
# echo 'if_tap_load="YES"' >> /boot/loader.conf
# echo 'xen_kernel="/boot/xen"' >> /boot/loader.conf
# echo 'xen_cmdline="dom0_mem=8192M dom0_max_vcpus=4 dom0=pvh console=com1,vga
com1=115200,8n1 guest_loglvl=all loglvl=all"' >> /boot/loader.conf
```



Log files that Xen™ creates for the DomU VMs are stored in /var/log/xen. Please be sure to check the contents of that directory if experiencing issues.

Activate the xencommons service during system startup:

```
# sysrc xencommons_enable=yes
```

These settings are enough to start a Dom0-enabled system. However, it lacks network functionality for the DomU machines. To fix that, define a bridged interface with the main NIC of the system which the DomU VMs can use to connect to the network. Replace *em0* with the host network interface name.

```
# sysrc cloned_interfaces="bridge0"
# sysrc ifconfig_bridge0="addm em0 SYNCDHCP"
# sysrc ifconfig_em0="up"
```

Restart the host to load the Xen™ kernel and start the Dom0.

```
# shutdown -r now
```

After successfully booting the Xen™ kernel and logging into the system again, the Xen™ management tool `xl` is used to show information about the domains.

```
# xl list
Name                                        ID   Mem VCPUs      State   Time(s)
Domain-0                                     0  8192     4     r-----     962.0
```

The output confirms that the Dom0 (called `Domain-0`) has the ID `0` and is running. It also has the memory and virtual CPUs that were defined in /boot/loader.conf earlier. More information can be found in the Xen™ Documentation. DomU guest VMs can now be created.

### 24.8.3. Xen™ DomU Guest VM Configuration

Unprivileged domains consist of a configuration file and virtual or physical hard disks. Virtual disk storage for the DomU can be files created by truncate(1) or ZFS volumes as described in "Creating and Destroying Volumes". In this example, a 20 GB volume is used. A VM is created with the ZFS volume, a FreeBSD ISO image, 1 GB of RAM and two virtual CPUs. The ISO installation file is retrieved with fetch(1) and saved locally in a file called freebsd.iso.

```
# fetch https://download.freebsd.org/releases/ISO-IMAGES/14.0/FreeBSD-14.0-RELEASE-
amd64-bootonly.iso -o freebsd.iso
```

A ZFS volume of 20 GB called xendisk0 is created to serve as the disk space for the VM.

```
# zfs create -V20G -o volmode=dev zroot/xendisk0
```

The new DomU guest VM is defined in a file. Some specific definitions like name, keymap, and VNC connection details are also defined. The following freebsd.cfg contains a minimum DomU configuration for this example:

```
# cat freebsd.cfg
builder = "hvm" ①
name = "freebsd" ②
memory = 1024 ③
vcpus = 2 ④
vif = [ 'mac=00:16:3E:74:34:32,bridge=bridge0' ] ⑤
disk = [
'/dev/zvol/tank/xendisk0,raw,hda,rw', ⑥
'/root/freebsd.iso,raw,hdc:cdrom,r' ⑦
  ]
vnc = 1 ⑧
vnclisten = "0.0.0.0"
serial = "pty"
```

```
usbdevice = "tablet"
```

These lines are explained in more detail:

① This defines what kind of virtualization to use. `hvm` refers to hardware-assisted virtualization or hardware virtual machine. Guest operating systems can run unmodified on CPUs with virtualization extensions, providing nearly the same performance as running on physical hardware. `generic` is the default value and creates a PV domain.

② Name of this virtual machine to distinguish it from others running on the same Dom0. Required.

③ Quantity of RAM in megabytes to make available to the VM. This amount is subtracted from the hypervisor's total available memory, not the memory of the Dom0.

④ Number of virtual CPUs available to the guest VM. For best performance, do not create guests with more virtual CPUs than the number of physical CPUs on the host.

⑤ Virtual network adapter. This is the bridge connected to the network interface of the host. The `mac` parameter is the MAC address set on the virtual network interface. This parameter is optional, if no MAC is provided Xen™ will generate a random one.

⑥ Full path to the disk, file, or ZFS volume of the disk storage for this VM. Options and multiple disk definitions are separated by commas.

⑦ Defines the Boot medium from which the initial operating system is installed. In this example, it is the ISO image downloaded earlier. Consult the Xen™ documentation for other kinds of devices and options to set.

⑧ Options controlling VNC connectivity to the serial console of the DomU. In order, these are: active VNC support, define IP address on which to listen, device node for the serial console, and the input method for precise positioning of the mouse and other input methods. `keymap` defines which keymap to use, and is `english` by default.

After the file has been created with all the necessary options, the DomU is created by passing it to `xl create` as a parameter.

```
# xl create freebsd.cfg
```



Each time the Dom0 is restarted, the configuration file must be passed to `xl create` again to re-create the DomU. By default, only the Dom0 is created after a reboot, not the individual VMs. The VMs can continue where they left off as they stored the operating system on the virtual disk. The virtual machine configuration can change over time (for example, when adding more memory). The virtual machine configuration files must be properly backed up and kept available to be able to re-create the guest VM when needed.

The output of `xl list` confirms that the DomU has been created.

```
# xl list
Name                                        ID   Mem VCPUs      State   Time(s)
Domain-0                                     0  8192     4     r-----  1653.4
```

```
freebsd                                  1  1024     1     -b----   663.9
```

To begin the installation of the base operating system, start the VNC client, directing it to the main network address of the host or to the IP address defined on the `vnclisten` line of freebsd.cfg. After the operating system has been installed, shut down the DomU and disconnect the VNC viewer. Edit freebsd.cfg, removing the line with the `cdrom` definition or commenting it out by inserting a `#` character at the beginning of the line. To load this new configuration, it is necessary to remove the old DomU with `xl destroy`, passing either the name or the id as the parameter. Afterwards, recreate it using the modified **freebsd.cfg**.

```
# xl destroy freebsd
# xl create freebsd.cfg
```

The machine can then be accessed again using the VNC viewer. This time, it will boot from the virtual disk where the operating system has been installed and can be used as a virtual machine.

## 24.8.4. Troubleshooting

This section contains basic information in order to help troubleshoot issues found when using FreeBSD as a Xen™ host or guest.

### 24.8.4.1. Host Boot Troubleshooting

Please note that the following troubleshooting tips are intended for Xen™ 4.11 or newer. When still using Xen™ 4.7 and having issues, consider migrating to a newer version of Xen™.

In order to troubleshoot host boot issues, a serial cable or a debug USB cable is needed. Verbose Xen™ boot output can be obtained by adding options to the `xen_cmdline` option found in loader.conf. A couple of relevant debug options are:

- `iommu=debug`: can be used to print additional diagnostic information about the iommu.
- `dom0=verbose`: can be used to print additional diagnostic information about the dom0 build process.
- `sync_console`: flag to force synchronous console output. Useful for debugging to avoid losing messages due to rate limiting. Never use this option in production environments since it can allow malicious guests to perform DoS attacks against Xen™ using the console.

FreeBSD should also be booted in verbose mode in order to identify any issues. To activate verbose booting, run this command:

```
# echo 'boot_verbose="YES"' >> /boot/loader.conf
```

If none of these options help solving the problem, please send the serial boot log to freebsd-xen@FreeBSD.org and xen-devel@lists.xenproject.org for further analysis.

#### 24.8.4.2. Guest Creation Troubleshooting

Issues can also arise when creating guests, the following attempts to provide some help for those trying to diagnose guest creation issues.

The most common cause of guest creation failures is the `xl` command spitting some error and exiting with a return code different than 0. If the error provided is not enough to help identify the issue, more verbose output can also be obtained from `xl` by using the `v` option repeatedly.

```
# xl -vvv create freebsd.cfg
Parsing config from freebsd.cfg
libxl: debug: libxl_create.c:1693:do_domain_create: Domain 0:ao 0x800d750a0: create:
how=0x0 callback=0x0 poller=0x800d6f0f0
libxl: debug: libxl_device.c:397:libxl__device_disk_set_backend: Disk vdev=xvda
spec.backend=unknown
libxl: debug: libxl_device.c:432:libxl__device_disk_set_backend: Disk vdev=xvda, using
backend phy
libxl: debug: libxl_create.c:1018:initiate_domain_create: Domain 1:running bootloader
libxl: debug: libxl_bootloader.c:328:libxl__bootloader_run: Domain 1:not a PV/PVH
domain, skipping bootloader
libxl: debug: libxl_event.c:689:libxl__ev_xswatch_deregister: watch w=0x800d96b98:
deregister unregistered
domainbuilder: detail: xc_dom_allocate: cmdline="", features=""
domainbuilder: detail: xc_dom_kernel_file: filename
="/usr/local/lib/xen/boot/hvmloader"
domainbuilder: detail: xc_dom_malloc_filemap    : 326 kB
libxl: debug: libxl_dom.c:988:libxl__load_hvm_firmware_module: Loading BIOS:
/usr/local/share/seabios/bios.bin
...
```

If the verbose output does not help diagnose the issue, there are also QEMU and Xen™ toolstack logs in /var/log/xen. Note that the name of the domain is appended to the log name, so if the domain is named `freebsd` find a /var/log/xen/xl-freebsd.log and likely a /var/log/xen/qemu-dm-freebsd.log. Both log files can contain useful information for debugging. If none of this helps solve the issue, please send the description of the issue and as much information as possible to freebsd-xen@FreeBSD.org and xen-devel@lists.xenproject.org to get help.

# Chapter 25. Localization - i18n/L10n Usage and Setup

## 25.1. Synopsis

FreeBSD is a distributed project with users and contributors located all over the world. As such, FreeBSD supports localization into many languages, allowing users to view, input, or process data in non-English languages. One can choose from most of the major languages, including, but not limited to: Chinese, German, Japanese, Korean, French, Russian, and Vietnamese.

The term internationalization has been shortened to i18n, which represents the number of letters between the first and the last letters of `internationalization`. L10n uses the same naming scheme, but from `localization`. The i18n/L10n methods, protocols, and applications allow users to use languages of their choice.

This chapter discusses the internationalization and localization features of FreeBSD.

Read this chapter to learn:

- How locale names are constructed.
- How to set the locale for a login shell.
- How to configure the console for non-English languages.
- How to configure Xorg for different languages.
- How to find i18n-compliant applications.
- Where to find more information for configuring specific languages.

Before reading this chapter:

- Know how to install additional third-party applications.

## 25.2. Using Localization

Localization settings are based on three components: the language code, country code, and encoding. Locale names are constructed from these parts as follows:

```
LanguageCode_CountryCode.Encoding
```

The *LanguageCode* and *CountryCode* are used to determine the country and the specific language variation. Common Language and Country Codes provides some examples of *LanguageCode_CountryCode*:

*Table 35. Common Language and Country Codes*

| LanguageCode_Country Code | Description |
|---|---|
| en_US | English, United States |
| ru_RU | Russian, Russia |
| zh_TW | Traditional Chinese, Taiwan |

A complete listing of available locales can be found by typing:

```
% locale -a | more
```

To determine the current locale setting:

```
% locale
```

Language specific character sets, such as ISO8859-1, ISO8859-15, KOI8-R, and CP437, are described in multibyte(3). The active list of character sets can be found at the IANA Registry.

Some languages, such as Chinese or Japanese, cannot be represented using ASCII characters and require an extended language encoding using either wide or multibyte characters. Examples of wide or multibyte encodings include EUC and Big5. Older applications may mistake these encodings for control characters while newer applications usually recognize these characters. Depending on the implementation, users may be required to compile an application with wide or multibyte character support, or to configure it correctly.



FreeBSD uses Xorg-compatible locale encodings.

The rest of this section describes the various methods for configuring the locale on a FreeBSD system. The next section will discuss the considerations for finding and compiling applications with i18n support.

### 25.2.1. Setting Locale for Login Shell

Locale settings are configured either in a user's ~/.login_conf or in the startup file of the user's shell: ~/.profile, ~/.bashrc, or ~/.cshrc.

Two environment variables should be set:

- `LANG`, which sets the locale
- `MM_CHARSET`, which sets the MIME character set used by applications

In addition to the user's shell configuration, these variables should also be set for specific application configuration and Xorg configuration.

Two methods are available for making the needed variable assignments: the login class method, which is the recommended method, and the startup file method. The next two sections demonstrate how to use both methods.

#### 25.2.1.1. Login Classes Method

This first method is the recommended method as it assigns the required environment variables for locale name and MIME character sets for every possible shell. This setup can either be performed by each user or it can be configured for all users by the superuser.

This minimal example sets both variables for Latin-1 encoding in the .login_conf of an individual user's home directory:

```
me:\
    :charset=ISO-8859-1:\
    :lang=de_DE.ISO8859-1:
```

Here is an example of a user's ~/.login_conf that sets the variables for Traditional Chinese in BIG-5 encoding. More variables are needed because some applications do not correctly respect locale variables for Chinese, Japanese, and Korean:

```
#Users who do not wish to use monetary units or time formats
#of Taiwan can manually change each variable
me:\
    :lang=zh_TW.Big5:\

:setenv=LC_ALL=zh_TW.Big5,LC_COLLATE=zh_TW.Big5,LC_CTYPE=zh_TW.Big5,LC_MESSAGES=zh_TW.
Big5,LC_MONETARY=zh_TW.Big5,LC_NUMERIC=zh_TW.Big5,LC_TIME=zh_TW.Big5:\
    :charset=big5:\
    :xmodifiers="@im=gcin": #Set gcin as the XIM Input Server
```

Alternately, the superuser can configure all users of the system for localization. The following variables in /etc/login.conf are used to set the locale and MIME character set:

```
language_name|Account Type Description:\
    :charset=MIME_charset:\
    :lang=locale_name:\
    :tc=default:
```

So, the previous Latin-1 example would look like this:

```
german|German Users Accounts:\
    :charset=ISO-8859-1:\
    :lang=de_DE.ISO8859-1:\
    :tc=default:
```

See login.conf(5) for more details about these variables. Note that it already contains pre-defined *russian* class.

Whenever /etc/login.conf is edited, remember to execute the following command to update the

capability database:

```
# cap_mkdb /etc/login.conf
```



For an end user, the `cap_mkdb` command will need to be run on their ~/.login_conf for any changes to take effect.

#### 25.2.1.1.1. Utilities Which Change Login Classes

In addition to manually editing /etc/login.conf, several utilities are available for setting the locale for newly created users.

When using `vipw` to add new users, specify the *language* to set the locale:

```
user:password:1111:11:language:0:0:User Name:/home/user:/bin/sh
```

When using `adduser` to add new users, the default language can be pre-configured for all new users or specified for an individual user.

If all new users use the same language, set `defaultclass=language` in /etc/adduser.conf.

To override this setting when creating a user, either input the required locale at this prompt:

```
Enter login class: default []:
```

or specify the locale to set when invoking `adduser`:

```
# adduser -class language
```

If `pw` is used to add new users, specify the locale as follows:

```
# pw useradd user_name -L language
```

To change the login class of an existing user, `chpass` can be used. Invoke it as superuser and provide the username to edit as the argument.

```
# chpass user_name
```

### 25.2.1.2. Shell Startup File Method

This second method is not recommended as each shell that is used requires manual configuration, where each shell has a different configuration file and differing syntax. As an example, to set the German language for the `sh` shell, these lines could be added to ~/.profile to set the shell for that

user only. These lines could also be added to /etc/profile or /usr/share/skel/dot.profile to set that shell for all users:

```
LANG=de_DE.ISO8859-1; export LANG
MM_CHARSET=ISO-8859-1; export MM_CHARSET
```

However, the name of the configuration file and the syntax used differs for the `csh` shell. These are the equivalent settings for ~/.login, /etc/csh.login, or /usr/share/skel/dot.login:

```
setenv LANG de_DE.ISO8859-1
setenv MM_CHARSET ISO-8859-1
```

To complicate matters, the syntax needed to configure Xorg in ~/.xinitrc also depends upon the shell. The first example is for the `sh` shell and the second is for the `csh` shell:

```
LANG=de_DE.ISO8859-1; export LANG
```

```
setenv LANG de_DE.ISO8859-1
```

### 25.2.2. Console Setup

Several localized fonts are available for the console. To see a listing of available fonts, type `ls /usr/share/syscons/fonts`. To configure the console font, specify the *font_name*, without the .fnt suffix, in /etc/rc.conf:

```
font8x16=font_name
font8x14=font_name
font8x8=font_name
```

The keymap and screenmap can be set by adding the following to /etc/rc.conf:

```
scrnmap=screenmap_name
keymap=keymap_name
keychange="fkey_number sequence"
```

To see the list of available screenmaps, type `ls /usr/share/syscons/scrnmaps`. Do not include the .scm suffix when specifying *screenmap_name*. A screenmap with a corresponding mapped font is usually needed as a workaround for expanding bit 8 to bit 9 on a VGA adapter's font character matrix so that letters are moved out of the pseudographics area if the screen font uses a bit 8 column.

To see the list of available keymaps, type `ls /usr/share/syscons/keymaps`. When specifying the *keymap_name*, do not include the .kbd suffix. To test keymaps without rebooting, use kbdmap(1).

The `keychange` entry is usually needed to program function keys to match the selected terminal type because function key sequences cannot be defined in the keymap.

Next, set the correct console terminal type in /etc/ttys for all virtual terminal entries. Defined Terminal Types for Character Sets summarizes the available terminal types.:

*Table 36. Defined Terminal Types for Character Sets*

| Character Set | Terminal Type |
|---|---|
| ISO8859-1 or ISO8859-15 | `cons25l1` |
| ISO8859-2 | `cons25l2` |
| ISO8859-7 | `cons25l7` |
| KOI8-R | `cons25r` |
| KOI8-U | `cons25u` |
| CP437 (VGA default) | `cons25` |
| US-ASCII | `cons25w` |

For languages with wide or multibyte characters, install a console for that language from the FreeBSD Ports Collection. The available ports are summarized in Available Console from Ports Collection. Once installed, refer to the port's pkg-message or man pages for configuration and usage instructions.

*Table 37. Available Console from Ports Collection*

| Language | Port Location |
|---|---|
| Traditional Chinese (BIG-5) | chinese/big5con |
| Chinese/Japanese/Korean | chinese/cce |
| Chinese/Japanese/Korean | chinese/zhcon |
| Japanese | chinese/kon2 |
| Japanese | japanese/kon2-14dot |
| Japanese | japanese/kon2-16dot |

If moused is enabled in /etc/rc.conf, additional configuration may be required. By default, the mouse cursor of the syscons(4) driver occupies the `0xd0-0xd3` range in the character set. If the language uses this range, move the cursor's range by adding the following line to /etc/rc.conf:

```
mousechar_start=3
```

### 25.2.3. Xorg Setup

The X Window System describes how to install and configure Xorg. When configuring Xorg for localization, additional fonts and input methods are available from the FreeBSD Ports Collection. Application specific i18n settings such as fonts and menus can be tuned in ~/.Xresources and should allow users to view their selected language in graphical application menus.

The X Input Method (XIM) protocol is an Xorg standard for inputting non-English characters. Available Input Methods summarizes the input method applications which are available in the FreeBSD Ports Collection. Additional Fcitx and Uim applications are also available.

*Table 38. Available Input Methods*

| Language | Input Method |
|---|---|
| Chinese | chinese/gcin |
| Chinese | chinese/ibus-chewing |
| Chinese | chinese/ibus-pinyin |
| Chinese | chinese/oxim |
| Chinese | chinese/scim-fcitx |
| Chinese | chinese/scim-pinyin |
| Chinese | chinese/scim-tables |
| Japanese | japanese/ibus-anthy |
| Japanese | japanese/ibus-mozc |
| Japanese | japanese/ibus-skk |
| Japanese | japanese/im-ja |
| Japanese | japanese/kinput2 |
| Japanese | japanese/scim-anthy |
| Japanese | japanese/scim-canna |
| Japanese | japanese/scim-honoka |
| Japanese | japanese/scim-honoka-plugin-romkan |
| Japanese | japanese/scim-honoka-plugin-wnn |
| Japanese | japanese/scim-prime |
| Japanese | japanese/scim-skk |
| Japanese | japanese/scim-tables |
| Japanese | japanese/scim-tomoe |
| Japanese | japanese/scim-uim |
| Japanese | japanese/skkinput |
| Japanese | japanese/skkinput3 |
| Japanese | japanese/uim-anthy |
| Korean | korean/ibus-hangul |
| Korean | korean/imhangul |
| Korean | korean/nabi |
| Korean | korean/scim-hangul |
| Korean | korean/scim-tables |

| Language | Input Method |
| --- | --- |
| Vietnamese | vietnamese/xvnkb |
| Vietnamese | vietnamese/x-unikey |

# 25.3. Finding i18n Applications

i18n applications are programmed using i18n kits under libraries. These allow developers to write a simple file and translate displayed menus and texts to each language.

The FreeBSD Ports Collection contains many applications with built-in support for wide or multibyte characters for several languages. Such applications include `i18n` in their names for easy identification. However, they do not always support the language needed.

Some applications can be compiled with the specific charset. This is usually done in the port's Makefile or by passing a value to configure. Refer to the i18n documentation in the respective FreeBSD port's source for more information on how to determine the needed configure value or the port's Makefile to determine which compile options to use when building the port.

# 25.4. Locale Configuration for Specific Languages

This section provides configuration examples for localizing a FreeBSD system for the Russian language. It then provides some additional resources for localizing other languages.

## 25.4.1. Russian Language (KOI8-R Encoding)

This section shows the specific settings needed to localize a FreeBSD system for the Russian language. Refer to Using Localization for a complete description of each type of setting.

To set this locale for the login shell, add the following lines to each user's ~/.login_conf:

```
me:My Account:\
    :charset=KOI8-R:\
    :lang=ru_RU.KOI8-R:
```

To configure the console, add the following lines to /etc/rc.conf:

```
keymap="ru.utf-8"
scrnmap="utf-82cp866"
font8x16="cp866b-8x16"
font8x14="cp866-8x14"
font8x8="cp866-8x8"
mousechar_start=3
```

For each `ttyv` entry in /etc/ttys, use `cons25r` as the terminal type.

To configure printing, a special output filter is needed to convert from KOI8-R to CP866 since most

printers with Russian characters come with hardware code page CP866. FreeBSD includes a default filter for this purpose, /usr/libexec/lpr/ru/koi2alt. To use this filter, add this entry to /etc/printcap:

```
lp|Russian local line printer:\
    :sh:of=/usr/libexec/lpr/ru/koi2alt:\
    :lp=/dev/lpt0:sd=/var/spool/output/lpd:lf=/var/log/lpd-errs:
```

Refer to printcap(5) for a more detailed explanation.

To configure support for Russian filenames in mounted MS-DOS® file systems, include `-L` and the locale name when adding an entry to /etc/fstab:

```
/dev/ad0s2      /dos/c  msdos   rw,-Lru_RU.KOI8-R 0 0
```

Refer to mount_msdosfs(8) for more details.

To configure Russian fonts for Xorg, install the x11-fonts/xorg-fonts-cyrillic package. Then, check the `"Files"` section in /etc/X11/xorg.conf. The following line must be added *before* any other `FontPath` entries:

```
FontPath   "/usr/local/lib/X11/fonts/cyrillic"
```

Additional Cyrillic fonts are available in the Ports Collection.

To activate a Russian keyboard, add the following to the `"Keyboard"` section of /etc/xorg.conf:

```
Option "XkbLayout"   "us,ru"
Option "XkbOptions"  "grp:toggle"
```

Make sure that `XkbDisable` is commented out in that file.

For `grp:toggle` use `Right Alt`, for `grp:ctrl_shift_toggle` use `Ctrl` + `Shift`. For `grp:caps_toggle` use `CapsLock`. The old `CapsLock` function is still available in LAT mode only using `Shift` + `CapsLock`. `grp:caps_toggle` does not work in Xorg for some unknown reason.

If the keyboard has "Windows®" keys, and some non-alphabetical keys are mapped incorrectly, add the following line to /etc/xorg.conf:

```
Option "XkbVariant" ",winkeys"
```



The Russian XKB keyboard may not work with non-localized applications. Minimally localized applications should call a `XtSetLanguageProc (NULL, NULL, NULL);` function early in the program.

See http://koi8.pp.ru/xwin.html for more instructions on localizing Xorg applications. For more general information about KOI8-R encoding, refer to http://koi8.pp.ru/.

### 25.4.2. Additional Language-Specific Resources

This section lists some additional resources for configuring other locales.

**Traditional Chinese for Taiwan**

The FreeBSD-Taiwan Project has a Chinese HOWTO for FreeBSD at http://netlab.cse.yzu.edu.tw/~statue/freebsd/zh-tut/.

**Greek Language Localization**

A complete article on Greek support in FreeBSD is available here, in Greek only, as part of the official FreeBSD Greek documentation.

**Japanese and Korean Language Localization**

For Japanese, refer to http://www.jp.FreeBSD.org/, and for Korean, refer to http://www.kr.FreeBSD.org/.

**Non-English FreeBSD Documentation**

Some FreeBSD contributors have translated parts of the FreeBSD documentation to other languages. They are available through links on the FreeBSD web site or in /usr/share/doc.

# Chapter 26. Updating and Upgrading FreeBSD

## 26.1. Synopsis

FreeBSD is under constant development between releases. Some people prefer to use the officially released versions, while others prefer to keep in sync with the latest developments. However, even official releases are often updated with security and other critical fixes. Regardless of the version used, FreeBSD provides all the necessary tools to keep the system updated, and allows for easy upgrades between versions. This chapter describes how to track the development system and the basic tools for keeping a FreeBSD system up-to-date.

Read this chapter to learn:

- How to keep a FreeBSD system up-to-date with freebsd-update or Git.
- How to compare the state of an installed system against a known pristine copy.
- How to keep the installed documentation up-to-date with Git or documentation ports.
- The difference between the two development branches: FreeBSD-STABLE and FreeBSD-CURRENT.
- How to rebuild and reinstall the entire base system.

Before reading this chapter:

- Properly set up the network connection (Advanced Networking).
- Know how to install additional third-party software (Installing Applications: Packages and Ports).



Throughout this chapter, `git` is used to obtain and update FreeBSD sources. Optionally, the devel/git port or package may be used.

## 26.2. FreeBSD Update

Applying security patches in a timely manner and upgrading to a newer release of an operating system are important aspects of ongoing system administration. FreeBSD includes a utility called `freebsd-update` which can be used to perform both these tasks.

This utility supports binary security and errata updates to FreeBSD, without the need to manually compile and install the patch or a new kernel. Binary updates are available for all architectures and releases currently supported by the security team. The list of supported releases and their estimated end-of-life dates are listed at https://www.FreeBSD.org/security/.

This utility also supports operating system upgrades to minor point releases as well as upgrades to another release branch. Before upgrading to a new release, review its release announcement as it contains important information pertinent to the release. Release announcements are available from https://www.FreeBSD.org/releases/.



If a crontab(5) utilizing the features of freebsd-update(8) exists, it must be disabled before upgrading the operating system.

This section describes the configuration file used by `freebsd-update`, demonstrates how to apply a security patch and how to upgrade to a minor or major operating system release, and discusses some of the considerations when upgrading the operating system.

### 26.2.1. The Configuration File

The default configuration file for `freebsd-update` works as-is. Some users may wish to tweak the default configuration in /etc/freebsd-update.conf, allowing better control of the process. The comments in this file explain the available options, but the following may require a bit more explanation:

```
# Components of the base system which should be kept updated.
Components world kernel
```

This parameter controls which parts of FreeBSD will be kept up-to-date. The default is to update the entire base system and the kernel. Individual components can instead be specified, such as `src/base` or `src/sys`. However, the best option is to leave this at the default as changing it to include specific items requires every needed item to be listed. Over time, this could have disastrous consequences as source code and binaries may become out of sync.

```
# Paths which start with anything matching an entry in an IgnorePaths
# statement will be ignored.
IgnorePaths /boot/kernel/linker.hints
```

To leave specified directories, such as /bin or /sbin, untouched during the update process, add their paths to this statement. This option may be used to prevent `freebsd-update` from overwriting local modifications.

```
# Paths which start with anything matching an entry in an UpdateIfUnmodified
# statement will only be updated if the contents of the file have not been
# modified by the user (unless changes are merged; see below).
UpdateIfUnmodified /etc/ /var/ /root/ /.cshrc /.profile
```

This option will only update unmodified configuration files in the specified directories. Any changes made by the user will prevent the automatic updating of these files. There is another option, `KeepModifiedMetadata`, which will instruct `freebsd-update` to save the changes during the merge.

```
# When upgrading to a new FreeBSD release, files which match MergeChanges
# will have any local changes merged into the version from the new release.
MergeChanges /etc/ /var/named/etc/ /boot/device.hints
```

List of directories with configuration files that `freebsd-update` should attempt to merge. The file merge process is a series of diff(1) patches. Merges are either accepted, open an editor, or cause `freebsd-update` to abort. When in doubt, backup /etc and just accept the merges.

```
# Directory in which to store downloaded updates and temporary
# files used by FreeBSD Update.
# WorkDir /var/db/freebsd-update
```

This directory is where all patches and temporary files are placed. In cases where the user is doing a version upgrade, this location should have at least a gigabyte of disk space available.

```
# When upgrading between releases, should the list of Components be
# read strictly (StrictComponents yes) or merely as a list of components
# which *might* be installed of which FreeBSD Update should figure out
# which actually are installed and upgrade those (StrictComponents no)?
# StrictComponents no
```

When this option is set to `yes`, `freebsd-update` will assume that the `Components` list is complete and will not attempt to make changes outside of the list. Effectively, `freebsd-update` will attempt to update every file which belongs to the `Components` list.

Refer to freebsd-update.conf(5) for more details.

### 26.2.2. Applying Security Patches

The process of applying FreeBSD security patches has been simplified, allowing an administrator to keep a system fully patched using `freebsd-update`. More information about FreeBSD security advisories can be found in FreeBSD Security Advisories.

FreeBSD security patches may be downloaded and installed using the following commands. The first command will determine if any outstanding patches are available, and if so, will list the files that will be modified if the patches are applied. The second command will apply the patches.

```
# freebsd-update fetch
# freebsd-update install
```

If the update applies any kernel patches, the system will need a reboot in order to boot into the patched kernel. If the patch was applied to any running binaries, the affected applications should be restarted so that the patched version of the binary is used.



Usually, the user needs to be prepared to reboot the system. To know if the system requires a reboot due to a kernel update, execute the commands `freebsd-version -k` and `uname -r`. Reboot the system if the outputs differ.

The system can be configured to automatically check for updates once every day by adding this entry to /etc/crontab:

```
@daily                                  root    freebsd-update cron
```

If patches exist, they will automatically be downloaded but will not be applied. The `root` user will be sent an email so that the patches may be reviewed and manually installed with `freebsd-update install`.

If anything goes wrong, `freebsd-update` has the ability to roll back the last set of changes with the following command:

```
# freebsd-update rollback
Uninstalling updates... done.
```

Again, the system should be restarted if the kernel or any kernel modules were modified and any affected binaries should be restarted.

Only the GENERIC kernel can be automatically updated by `freebsd-update`. If a custom kernel is installed, it will have to be rebuilt and reinstalled after `freebsd-update` finishes installing the updates. The default kernel name is *GENERIC*. The uname(1) command may be used to verify its installation.



Always keep a copy of the GENERIC kernel in /boot/GENERIC. It will be helpful in diagnosing a variety of problems and in performing version upgrades. Refer to Custom Kernels with FreeBSD 9.X and Later for instructions on how to get a copy of the GENERIC kernel.

Unless the default configuration in /etc/freebsd-update.conf has been changed, `freebsd-update` will install the updated kernel sources along with the rest of the updates. Rebuilding and reinstalling a new custom kernel can then be performed in the usual way.

The updates distributed by `freebsd-update` do not always involve the kernel. It is not necessary to rebuild a custom kernel if the kernel sources have not been modified by `freebsd-update install`. However, `freebsd-update` will always update /usr/src/sys/conf/newvers.sh. The current patch level, as indicated by the `-p` number reported by `uname -r`, is obtained from this file. Rebuilding a custom kernel, even if nothing else changed, allows `uname` to accurately report the current patch level of the system. This is particularly helpful when maintaining multiple systems, as it allows for a quick assessment of the updates installed in each one.

### 26.2.3. Performing Minor and Major Version Upgrades

Upgrades from one minor version of FreeBSD to another are called *minor version* upgrades. An example:

- FreeBSD 13.1 to 13.2.

*Major version* upgrades increase the major version number. An example:

- FreeBSD 13.2 to 14.0.

Both types of upgrade can be performed by providing `freebsd-update` with a release version target.



After each new `RELEASE`, FreeBSD package build servers will, for a limited period, **not** use the newer version of the operating system. This provides continuity for the many users who do not upgrade immediately after a release announcement. For example:

- packages for users of 13.1 and 13.2 will be built on a server running 13.1, until 13.1 reaches end of life

— and, critically:

- a kernel module that is built on 13.1 might **not** be suitable for 13.2.

So, with any minor or major OS upgrade, if the package requirements include any kernel module:

- **be prepared to build the module from source**.



If the system is running a custom kernel, make sure that a copy of the GENERIC kernel exists in /boot/GENERIC before starting the upgrade. Refer to Custom Kernels with FreeBSD 9.X and Later for instructions on how to get a copy of the GENERIC kernel.

Before upgrading to a new version, ensure the existing FreeBSD installation is up to date with respect to security and errata patches:

```
# freebsd-update fetch
# freebsd-update install
```

The following command, when run on a FreeBSD 13.1 system, will upgrade it to FreeBSD 13.2:

```
# freebsd-update -r 13.2-RELEASE upgrade
```

After the command has been received, `freebsd-update` will evaluate the configuration file and current system in an attempt to gather the information necessary to perform the upgrade. A screen listing will display which components have and have not been detected. For example:

```
Looking up update.FreeBSD.org mirrors... 1 mirrors found.
Fetching metadata signature for 13.1-RELEASE from update1.FreeBSD.org... done.
Fetching metadata index... done.
Inspecting system... done.

The following components of FreeBSD seem to be installed:
kernel/smp src/base src/bin src/contrib src/crypto src/etc src/games
src/gnu src/include src/krb5 src/lib src/libexec src/release src/rescue
src/sbin src/secure src/share src/sys src/tools src/ubin src/usbin
```

```
world/base world/info world/lib32 world/manpages

The following components of FreeBSD do not seem to be installed:
kernel/generic world/catpages world/dict world/doc world/games
world/proflibs

Does this look reasonable (y/n)? y
```

At this point, `freebsd-update` will attempt to download all files required for the upgrade. In some cases, the user may be prompted with questions regarding what to install or how to proceed.

When using a custom kernel, the above step will produce a warning similar to the following:

```
WARNING: This system is running a "MYKERNEL" kernel, which is not a
kernel configuration distributed as part of FreeBSD 13.1-RELEASE.
This kernel will not be updated: you MUST update the kernel manually
before running "/usr/sbin/freebsd-update install"
```

This warning may be safely ignored at this point. The updated GENERIC kernel will be used as an intermediate step in the upgrade process.

Once all the patches have been downloaded to the local system, they will be applied. This process may take a while, depending on the speed and workload of the machine. Configuration files will then be merged. The merging process requires some user intervention as a file may be merged or an editor may appear on screen for a manual merge. The results of every successful merge will be shown to the user as the process continues. A failed or ignored merge will cause the process to abort. Users may wish to make a backup of /etc and manually merge important files, such as master.passwd or group at a later time.



The system is not being altered yet as all patching and merging is happening in another directory. Once all patches have been applied successfully, all configuration files have been merged and it seems the process will go smoothly, the changes can be committed to disk by the user using the following command:

```
# freebsd-update install
```

The kernel and kernel modules will be patched first. If the system is running with a custom kernel, use nextboot(8) to set the kernel for the next boot to the updated /boot/GENERIC:

```
# nextboot -k GENERIC
```



Before rebooting with the GENERIC kernel, make sure it contains all the drivers required for the system to boot properly and connect to the network, if the machine being updated is accessed remotely. In particular, if the running custom kernel contains built-in functionality usually provided by kernel modules, make

sure to temporarily load these modules into the GENERIC kernel using the /boot/loader.conf facility. It is recommended to disable non-essential services as well as any disk and network mounts until the upgrade process is complete.

The machine should now be restarted with the updated kernel:

```
# shutdown -r now
```

Once the system has come back online, restart `freebsd-update` using the following command. Since the state of the process has been saved, `freebsd-update` will not start from the beginning, but will instead move on to the next phase and remove all old shared libraries and object files.

```
# freebsd-update install
```



Depending upon whether any library version numbers were bumped, there may only be two install phases instead of three.

The upgrade is now complete. If this was a major version upgrade, reinstall all ports and packages as described in Upgrading Packages After a Major Version Upgrade.

#### 26.2.3.1. Custom Kernels with FreeBSD 9.X and Later

Before using `freebsd-update`, ensure that a copy of the GENERIC kernel exists in /boot/GENERIC. If a custom kernel has only been built once, the kernel in /boot/kernel.old is the `GENERIC` kernel. Simply rename this directory to /boot/GENERIC.

If a custom kernel has been built more than once or if it is unknown how many times the custom kernel has been built, obtain a copy of the `GENERIC` kernel that matches the current version of the operating system. If physical access to the system is available, a copy of the `GENERIC` kernel can be installed from the installation media:

```
# mount /media
# cd /media/usr/freebsd-dist
# tar -C/ -xvf kernel.txz boot/kernel/kernel
```

Alternately, the `GENERIC` kernel may be rebuilt and installed from source:

```
# cd /usr/src
# make kernel __MAKE_CONF=/dev/null SRCCONF=/dev/null
```

For this kernel to be identified as the `GENERIC` kernel by `freebsd-update`, the GENERIC configuration file must not have been modified in any way. It is also suggested that the kernel is built without any other special options.

Rebooting into the GENERIC kernel is not required as `freebsd-update` only needs /boot/GENERIC to

exist.

#### 26.2.3.2. Upgrading Packages After a Major Version Upgrade

Generally, installed applications will continue to work without problems after minor version upgrades. Major versions use different Application Binary Interfaces (ABIs), which will break most third-party applications. After a major version upgrade, all installed packages and ports need to be upgraded. Packages can be upgraded using `pkg upgrade`. To upgrade installed ports, use a utility such as ports-mgmt/portmaster.

A forced upgrade of all installed packages will replace the packages with fresh versions from the repository even if the version number has not increased. This is required because of the ABI version change when upgrading between major versions of FreeBSD. The forced upgrade can be accomplished by performing:

```
# pkg-static upgrade -f
```

A rebuild of all applications installed from the ports collection can be accomplished with this command:

```
# portmaster -af
```

This command will display the configuration screens for each application that has configurable options and wait for the user to interact with those screens. To prevent this behavior, and use only the default options, include `-G` in the above command.

Once the software upgrades are complete, finish the upgrade process with a final call to `freebsd-update` in order to tie up all the loose ends in the upgrade process:

```
# freebsd-update install
```

If the GENERIC kernel was temporarily used, this is the time to build and install a new custom kernel using the instructions in Configuring the FreeBSD Kernel.

Reboot the machine into the new FreeBSD version. The upgrade process is now complete.

### 26.2.4. System State Comparison

The state of the installed FreeBSD version against a known good copy can be tested using `freebsd-update IDS`. This command evaluates the current version of system utilities, libraries, and configuration files and can be used as a built-in Intrusion Detection System (IDS).



This command is not a replacement for a real IDS such as security/snort. As `freebsd-update` stores data on disk, the possibility of tampering is evident. While this possibility may be reduced using `kern.securelevel` and by storing the `freebsd-update` data on a read-only file system when not in use, a better solution would be

> to compare the system against a secure disk, such as a DVD or securely stored external USB disk device. An alternative method for providing IDS functionality using a built-in utility is described in Binary Verification

To begin the comparison, specify the output file to save the results to:

```
# freebsd-update IDS >> outfile.ids
```

The system will now be inspected and a lengthy listing of files, along with the SHA256 hash values for both the known value in the release and the current installation, will be sent to the specified output file.

The entries in the listing are extremely long, but the output format may be easily parsed. For instance, to obtain a list of all files which differ from those in the release, issue the following command:

```
# cat outfile.ids | awk '{ print $1 }' | more
/etc/master.passwd
/etc/motd
/etc/passwd
/etc/pf.conf
```

This sample output has been truncated as many more files exist. Some files have natural modifications. For example, /etc/passwd will be modified if users have been added to the system. Kernel modules may differ as `freebsd-update` may have updated them. To exclude specific files or directories, add them to the `IDSIgnorePaths` option in /etc/freebsd-update.conf.

## 26.3. Updating Bootcode

The following manuals describe the upgrade process of bootcode and boot loaders: gpart(8), gptboot(8), gptzfsboot(8), and loader.efi(8).

## 26.4. Updating the Documentation Set

Documentation is an integral part of the FreeBSD operating system. While an up-to-date version of the FreeBSD documentation is always available on the FreeBSD web site (Documentation Portal), it can be handy to have an up-to-date, local copy of the FreeBSD website, handbooks, FAQ, and articles.

This section describes how to use either source or the FreeBSD Ports Collection to keep a local copy of the FreeBSD documentation up-to-date.

For information on editing and submitting corrections to the documentation, refer to the FreeBSD Documentation Project Primer for New Contributors (FreeBSD Documentation Project Primer for New Contributors).

### 26.4.1. Updating Documentation from Source

Rebuilding the FreeBSD documentation from source requires a collection of tools which are not part of the FreeBSD base system. The required tools can be installed following these steps from the FreeBSD Documentation Project Primer.

Once installed, use `git` to fetch a clean copy of the documentation source:

```
# git clone https://git.FreeBSD.org/doc.git /usr/doc
```

The initial download of the documentation sources may take a while. Let it run until it completes.

Future updates of the documentation sources may be fetched by running:

```
# git pull
```

Once an up-to-date snapshot of the documentation sources has been fetched to /usr/doc, everything is ready for an update of the installed documentation.

A full update may be performed by typing:

```
# cd /usr/doc
# make
```

## 26.5. Tracking a Development Branch

FreeBSD has two development branches: FreeBSD-CURRENT and FreeBSD-STABLE.

This section provides an explanation of each branch and its intended audience, as well as how to keep a system up-to-date with each respective branch.

### 26.5.1. Using FreeBSD-CURRENT

FreeBSD-CURRENT is the "bleeding edge" of FreeBSD development and FreeBSD-CURRENT users are expected to have a high degree of technical skill. Less technical users who wish to track a development branch should track FreeBSD-STABLE instead.

FreeBSD-CURRENT is the very latest source code for FreeBSD and includes works in progress, experimental changes, and transitional mechanisms that might or might not be present in the next official release. While many FreeBSD developers compile the FreeBSD-CURRENT source code daily, there are short periods of time when the source may not be buildable. These problems are resolved as quickly as possible, but whether or not FreeBSD-CURRENT brings disaster or new functionality can be a matter of when the source code was synced.

FreeBSD-CURRENT is made available for three primary interest groups:

1. Members of the FreeBSD community who are actively working on some part of the source tree.

2. Members of the FreeBSD community who are active testers. They are willing to spend time solving problems, making topical suggestions on changes and the general direction of FreeBSD, and submitting patches.
3. Users who wish to keep an eye on things, use the current source for reference purposes, or make the occasional comment or code contribution.

FreeBSD-CURRENT should *not* be considered a fast-track to getting new features before the next release as pre-release features are not yet fully tested and most likely contain bugs. It is not a quick way of getting bug fixes as any given commit is just as likely to introduce new bugs as to fix existing ones. FreeBSD-CURRENT is not in any way "officially supported".

To track FreeBSD-CURRENT:

1. Join the FreeBSD-CURRENT mailing list and the Commit messages for the main branch of the src repository lists. This is *essential* in order to see the comments that people are making about the current state of the system and to receive important bulletins about the current state of FreeBSD-CURRENT.

   The Commit messages for the main branch of the src repository list records the commit log entry for each change as it is made, along with any pertinent information on possible side effects.

   To join these lists, go to FreeBSD list server, click on the list to subscribe to, and follow the instructions. In order to track changes to the whole source tree, not just the changes to FreeBSD-CURRENT, subscribe to the Commit messages for all branches of the src repository.

2. Synchronize with the FreeBSD-CURRENT sources. Typically, `git` is used to check out the -CURRENT code from the `main` branch of the FreeBSD Git repository (see “Using Git” for details).
3. Due to the size of the repository, some users choose to only synchronize the sections of source that interest them or which they are contributing patches to. However, users that plan to compile the operating system from source must download *all* of FreeBSD-CURRENT, not just selected portions.

   Before compiling FreeBSD-CURRENT, read /usr/src/Makefile very carefully and follow the instructions in Updating FreeBSD from Source. Read the FreeBSD-CURRENT mailing list and /usr/src/UPDATING to stay up-to-date on other bootstrapping procedures that sometimes become necessary on the road to the next release.

4. Be active! FreeBSD-CURRENT users are encouraged to submit their suggestions for enhancements or bug fixes. Suggestions with accompanying code are always welcome.

### 26.5.2. Using FreeBSD-STABLE

FreeBSD-STABLE is the development branch from which major releases are made. Changes go into this branch at a slower pace and with the general assumption that they have first been tested in FreeBSD-CURRENT. This is *still* a development branch and, at any given time, the sources for FreeBSD-STABLE may or may not be suitable for general use. It is simply another engineering development track, not a resource for end-users. Users who do not have the resources to perform testing should instead run the most recent release of FreeBSD.

Those interested in tracking or contributing to the FreeBSD development process, especially as it relates to the next release of FreeBSD, should consider following FreeBSD-STABLE.

While the FreeBSD-STABLE branch should compile and run at all times, this cannot be guaranteed. Since more people run FreeBSD-STABLE than FreeBSD-CURRENT, it is inevitable that bugs and corner cases will sometimes be found in FreeBSD-STABLE that were not apparent in FreeBSD-CURRENT. For this reason, one should not blindly track FreeBSD-STABLE. It is particularly important *not* to update any production servers to FreeBSD-STABLE without thoroughly testing the code in a development or testing environment.

To track FreeBSD-STABLE:

1. Join the FreeBSD-STABLE mailing list in order to stay informed of build dependencies that may appear in FreeBSD-STABLE or any other issues requiring special attention. Developers will also make announcements in this mailing list when they are contemplating some controversial fix or update, giving the users a chance to respond if they have any issues to raise concerning the proposed change.

   Join the relevant git list for the branch being tracked. For example, users tracking the 14-STABLE branch should join the Commit messages for the stable branches of the src repository. This list records the commit log entry for each change as it is made, along with any pertinent information on possible side effects.

   To join these lists, go to FreeBSD list server, click on the list to subscribe to, and follow the instructions. In order to track changes for the whole source tree, subscribe to Commit messages for all branches of the src repository.

2. To install a new FreeBSD-STABLE system, install the most recent FreeBSD-STABLE release from the FreeBSD mirror sites or use a monthly snapshot built from FreeBSD-STABLE. Refer to www.freebsd.org/snapshots for more information about snapshots.

   To compile or upgrade an existing FreeBSD system to FreeBSD-STABLE, use `git` to check out the source for the desired branch. Branch names, such as `stable/13`, are listed at www.freebsd.org/releng.

3. Before compiling or upgrading to FreeBSD-STABLE , read /usr/src/Makefile carefully and follow the instructions in Updating FreeBSD from Source. Read the FreeBSD-STABLE mailing list and /usr/src/UPDATING to keep up-to-date on other bootstrapping procedures that sometimes become necessary on the road to the next release.

### 26.5.3. The N-number

When tracking down bugs it is important to know which versions of the source code have been used to create the system exhibiting an issue. FreeBSD provides version information compiled into the kernel. uname(1) retrieves this information, for example:

```
% uname -v
FreeBSD 14.0-CURRENT #112 main-n247514-031260d64c18: Tue Jun 22 20:43:19 MDT 2021
fred@machine:/usr/home/fred/obj/usr/home/fred/git/head/amd64.amd64/sys/FRED
```

The final field gives information regarding the kernel name, the person that built it, and the location that it was compiled in. Looking at the 4th field, it is made up of several parts:

```
main-n247514-031260d64c18

main        ①
n247514     ②
031260d64c18    ③
④
```

① Git branch name. Note: comparisons of n-numbers are only valid on branches published by the project (`main`, `stable/XX` and `releng/XX`). Local branches will have n-numbers that will overlap commits of their parent branch.

② The n-number is a linear count of commits back to the start of the Git repository starting with the Git hash included in the line.

③ Git hash of the checked out tree

④ Sometimes a suffix of `-dirty` is present when the kernel was built in a tree with uncommitted changes. It is absent in this example because the FRED kernel was built from a pristine checkout.

The `git rev-list` command is used to find the n-number corresponding to a Git hash. For example:

```
% git rev-list --first-parent --count 031260d64c18 ①
247514 ②
```

① git hash to translate (the hash from the above example is reused)

② The n-number.

Usually this number is not all that important. However, when bug fixes are committed, this number makes it easy to quickly determine whether the fix is present in the currently running system. Developers will often refer to the hash of the commit (or provide a URL which has that hash), but not the n-number since the hash is the easily visible identifier for a change while the n-number is not. Security advisories and errata notices will also note an n-number, which can be directly compared against the system. When shallow Git clones are used, n-numbers cannot be compared reliably, as the git rev-list command counts all revisions in the repository, which a shallow clone omits.

## 26.6. Updating FreeBSD from Source

Updating FreeBSD by compiling from source offers several advantages over binary updates. Code can be built with options to take advantage of specific hardware. Parts of the base system can be built with non-default settings, or left out entirely where they are not needed or desired. The build process takes longer to update a system than just installing binary updates, but allows complete customization to produce a tailored version of FreeBSD.

### 26.6.1. Quick Start

This is a quick reference for the typical steps used to update FreeBSD by building from source. Later sections describe the process in more detail.



When switching from mergemaster(8) to etcupdate(8), the first run might merge changes incorrectly generating spurious conflicts. To prevent this, perform the following steps **before** updating sources and building the new world:

```
# etcupdate extract ①
# etcupdate diff ②
```

① Bootstrap the database of stock /etc files; for more information see etcupdate(8).

② Check the diff after bootstrapping. Trim any local changes that are no longer needed to reduce the chance of conflicts in future updates.

- Update and Build

```
# git -C /usr/src pull ①
check /usr/src/UPDATING  ②
# cd /usr/src           ③
# make -j4 buildworld   ④
# make -j4 kernel       ⑤
# shutdown -r now       ⑥
# etcupdate -p          ⑦
# cd /usr/src           ⑧
# make installworld     ⑨
# etcupdate -B          ⑩
# shutdown -r now       ⑪
```

① Get the latest version of the source. See Updating the Source for more information on obtaining and updating source.

② Check /usr/src/UPDATING for any manual steps required before or after building from source.

③ Go to the source directory.

④ Compile the world, everything except the kernel.

⑤ Compile and install the kernel. This is equivalent to `make buildkernel installkernel`.

⑥ Reboot the system to the new kernel.

⑦ Update and merge configuration files in /etc/ required before installworld.

⑧ Go to the source directory.

⑨ Install the world.

⑩ Update and merge configuration files in /etc/.

⑪ Restart the system to use the newly-built world and kernel.

### 26.6.2. Preparing for a Source Update

Read /usr/src/UPDATING. Any manual steps that must be performed before or after an update are described in this file.

### 26.6.3. Updating the Source

FreeBSD source code is located in /usr/src/. The preferred method of updating this source is through the Git version control system. Verify that the source code is under version control:

```
# cd /usr/src
# git remote --v
origin  https://git.freebsd.org/src.git (fetch)
origin  https://git.freebsd.org/src.git (push)
```

This indicates that /usr/src/ is under version control and can be updated with git(1):

```
# git -C /usr/src pull
```

The update process can take some time if the directory has not been updated recently. After it finishes, the source code is up to date and the build process described in the next section can begin.



Obtaining the source:

If the output says `fatal: not a git repository`, the files there are missing or were installed with a different method. A new checkout of the source is required.

*Table 39. FreeBSD Versions and Repository Branches*

| uname -r Output | Repository Path | Description |
|---|---|---|
| `X.Y-RELEASE` | `releng/X.Y` | The Release version plus only critical security and bug fix patches. This branch is recommended for most users. |
| `X.Y-STABLE` | `stable/X` | The Release version plus all additional development on that branch. *STABLE* refers to the Applications Binary Interface (ABI) not changing, so software compiled for earlier versions still runs. For example, software compiled to run on FreeBSD 14.4 will still run on FreeBSD 14-STABLE compiled later.<br><br>STABLE branches occasionally have bugs or incompatibilities which might affect users, although these are typically fixed quickly. |

| uname -r Output | Repository Path | Description |
|---|---|---|
| X-CURRENT | main | The latest unreleased development version of FreeBSD. The CURRENT branch can have major bugs or incompatibilities and is recommended only for advanced users. |

Determine which version of FreeBSD is being used with uname(1):

```
# uname -r
13.2-RELEASE
```

Based on FreeBSD Versions and Repository Branches, the source used to update 13.2-RELEASE has a repository path of releng/13.2. That path is used when checking out the source:

```
# mv /usr/src /usr/src.bak ①
# git clone --branch releng/13.2 https://git.FreeBSD.org/src.git /usr/src ②
```

① Move the old directory out of the way. If there are no local modifications in this directory, it can be deleted.

② The path from FreeBSD Versions and Repository Branches is added to the repository URL. The third parameter is the destination directory for the source code on the local system.

## 26.6.4. Building from Source

The *world*, or all of the operating system except the kernel, is compiled. This is done first to provide up-to-date tools to build the kernel. Then the kernel itself is built:

```
# cd /usr/src
# make buildworld
# make buildkernel
```

The compiled code is written to /usr/obj.

These are the basic steps. Additional options to control the build are described below.

### 26.6.4.1. Performing a Clean Build

Some versions of the FreeBSD build system leave previously-compiled code in the temporary object directory, /usr/obj. This can speed up later builds by avoiding recompiling code that has not changed. To force a clean rebuild of everything, use cleanworld before starting a build:

```
# make cleanworld
```

#### 26.6.4.2. Setting the Number of Jobs

Increasing the number of build jobs on multi-core processors can improve build speed. Determine the number of cores with `sysctl hw.ncpu`. Processors vary, as do the build systems used with different versions of FreeBSD, so testing is the only sure method to tell how a different number of jobs affects the build speed. For a starting point, consider values between half and double the number of cores. The number of jobs is specified with `-j`.

*Example 39. Increasing the Number of Build Jobs*

Building the world and kernel with four jobs:

```
# make -j4 buildworld buildkernel
```

#### 26.6.4.3. Building Only the Kernel

A `buildworld` must be completed if the source code has changed. After that, a `buildkernel` to build a kernel can be run at any time. To build just the kernel:

```
# cd /usr/src
# make buildkernel
```

#### 26.6.4.4. Building a Custom Kernel

The standard FreeBSD kernel is based on a *kernel config file* called GENERIC. The GENERIC kernel includes the most commonly-needed device drivers and options. Sometimes it is useful or necessary to build a custom kernel, adding or removing device drivers or options to fit a specific need.

For example, someone developing a small embedded computer with severely limited RAM could remove unneeded device drivers or options to make the kernel slightly smaller.

Kernel config files are located in /usr/src/sys/arch/conf/, where *arch* is the output from `uname -m`. On most computers, that is `amd64`, giving a config file directory of /usr/src/sys/amd64/conf/.



/usr/src can be deleted or recreated, so it is preferable to keep custom kernel config files in a separate directory, like /root. Link the kernel config file into the conf directory. If that directory is deleted or overwritten, the kernel config can be re-linked into the new one.

A custom config file can be created by copying the GENERIC config file. In this example, the new custom kernel is for a storage server, so is named STORAGESERVER:

```
# cp /usr/src/sys/amd64/conf/GENERIC /root/STORAGESERVER
# cd /usr/src/sys/amd64/conf
# ln -s /root/STORAGESERVER .
```

/root/STORAGESERVER is then edited, adding or removing devices or options as shown in config(5).

The custom kernel is built by setting KERNCONF to the kernel config file on the command line:

```
# make buildkernel KERNCONF=STORAGESERVER
```

### 26.6.5. Installing the Compiled Code

After the buildworld and buildkernel steps have been completed, the new kernel and world are installed:

```
# cd /usr/src
# make installkernel
# shutdown -r now
# cd /usr/src
# make installworld
# shutdown -r now
```

If a custom kernel was built, KERNCONF must also be set to use the new custom kernel:

```
# cd /usr/src
# make installkernel KERNCONF=STORAGESERVER
# shutdown -r now
# cd /usr/src
# make installworld
# shutdown -r now
```

### 26.6.6. Completing the Update

A few final tasks complete the update. Any modified configuration files are merged with the new versions, outdated libraries are located and removed, then the system is restarted.

#### 26.6.6.1. Merging Configuration Files with etcupdate(8)

etcupdate(8) is a tool for managing updates to files that are not updated as part of an installworld such as files located in /etc/. It manages updates by doing a three-way merge of changes made to these files against the local versions. etcupdate(8) is designed to minimize the amount of user intervention.



In general, etcupdate(8) does not need any specific arguments for its job. There is however a handy in between command for sanity checking what will be done the first time etcupdate(8) is used:

```
# etcupdate diff
```

This command allows the user to audit configuration changes.

If etcupdate(8) is not able to merge a file automatically, the merge conflicts can be resolved with manual interaction by issuing:

```
# etcupdate resolve
```



When switching from mergemaster(8) to etcupdate(8), the first run might merge changes incorrectly generating spurious conflicts. To prevent this, perform the following steps **before** updating sources and building the new world:

```
# etcupdate extract ①
# etcupdate diff ②
```

① Bootstrap the database of stock /etc files; for more information see etcupdate(8).

② Check the diff after bootstrapping. Trim any local changes that are no longer needed to reduce the chance of conflicts in future updates.

#### 26.6.6.2. Checking for Outdated Files and Libraries

Some obsolete files or directories can remain after an update. These files can be located:

```
# make check-old
```

and deleted:

```
# make delete-old
```

Some obsolete libraries can also remain. These can be detected with:

```
# make check-old-libs
```

and deleted with

```
# make delete-old-libs
```

Programs which were still using those old libraries will stop working when the library has been deleted. These programs must be rebuilt or replaced after deleting the old libraries.



When all the old files or directories are known to be safe to delete, pressing `y` and `Enter` to delete each file can be avoided by setting `BATCH_DELETE_OLD_FILES` in the command. For example:

```
# make BATCH_DELETE_OLD_FILES=yes delete-old-libs
```

#### 26.6.6.3. Restarting After the Update

The last step after updating is to restart the computer so all the changes take effect:

```
# shutdown -r now
```

## 26.7. Updating FreeBSD with packages

Starting from 14.0-RELEASE, the FreeBSD project has published a set of packages of the base system, using pkg(8). The convenience of pkg is no longer limited to the FreeBSD ports collection. pkg(8) can be used for FreeBSD itself. Base packages, and usage thereof, were traditionally referred to as *pkgbase*.

Publication of official packages began in October 2023. Their use with FreeBSD 14 is currently experimental.

Starting from 15.0-RELEASE, freebsd-base(7) will be supported as a tech preview for:

- installation
- minor upgrades
- major upgrades

freebsd-base will eventually replace:

- tarball distribution sets, such as `base.txz` or `kernel.txz`, which are historically used for installation of the OS with bsdinstall(8)
- freebsd-update(8) for updates to the OS.

The current plan is for freebsd-update to work with pkg.



The replacement of distribution sets will take some time, until freebsd-base is fully stable!

Base System packages complement building and installing from source, which is still available for those who wish to build their own custom kernels or userland. It is also possible to build custom base system packages from local sources, as well as just relying on officially provided packages.

### 26.7.1. Converting a Host to use freebsd-base

FreeBSD 14.0-RELEASE or later can be converted to use base packages. Earlier versions must be upgraded with freebsd-update before conversion.

For conversion, the FreeBSD Foundation-sponsored pkgbasify tool is recommended. Follow the instructions given there.

freebsd-update(8) will be enhanced to use pkg and pkgbasify.



Note that this migration requires up to 5GiB additional free space, to download, unpack, and relocate any conflicting files. The pkgbasify tool does not check for this and it is the responsibility of the user to ensure that enough space is available before running the migration.

pkgbasify(8) (or whatever outcome D51594 or port request will have) performs 6 main tasks:

- Creates a backup boot environment (ZFS only) with bectl(8)
- Creates the new package repository config files
- Upgrades existing system components such as base, kernel, lib32, debug
- Merges existing and new config files
- Updates passwd and capabilities databases
- Restarts sshd immediately

### 26.7.2. Upgrading a Host using freebsd-base



This is still in development, so please be careful especially when converting an existing system to use freebsd-base.

Create a folder for custom pkg repository config files, if there is none present already.

```
# mkdir -p /usr/local/etc/pkg/repos/
```

For using the freebsd-base repository, create a pkg repository configuration file called `FreeBSD-base.conf`:

```
FreeBSD-base {
    url = "pkg+https://pkg.freebsd.org/${ABI}/base_release_${VERSION_MINOR}";
    mirror_type = "srv";
    signature_type = "fingerprints";
    fingerprints = "/usr/share/keys/pkg";
    enabled = yes;
}
```

for more information on specific configuration options see pkg.conf(5).

There are different branches to choose from (by changing the url accordingly):

*Table 40. Base system package Branches*

| Branch | Frequency | URL |
|---|---|---|
| main | twice daily - 12:00 and 00:00 UTC | `pkg+https://pkg.freebsd.org/${ABI}/base_latest` |

| | | |
|---|---|---|
| main | weekly – Sunday at 12:00 UTC | pkg+https://pkg.freebsd.org/${ABI}/base_weekly |
| stable/14 | twice daily – 12:00 and 00:00 UTC | pkg+https://pkg.freebsd.org/${ABI}/base_latest |
| stable/14 | weekly – Sunday at 12:00 UTC | pkg+https://pkg.freebsd.org/${ABI}/base_weekly |
| releng/14.3 | twice daily – 12:00 and 00:00 UTC | pkg+https://pkg.freebsd.org/${ABI}/base_release_3 |
| releng/15.0 | twice daily – 12:00 and 00:00 UTC | pkg+https://pkg.freebsd.org/${ABI}/base_release_0 |
| stable/15 | twice daily – 12:00 and 00:00 UTC | pkg+https://pkg.freebsd.org/${ABI}/base_latest |
| stable/15 | weekly – Sunday at 12:00 UTC | pkg+https://pkg.freebsd.org/${ABI}/base_weekly |

To upgrade the system, change the configuration file according to the desired release, and run:

```
# pkg update -r FreeBSD-base
# pkg upgrade -r FreeBSD-base
```

check, if these packages are correct and accept the changes.

Reboot the OS executing the following command:

```
# shutdown -r now
```

#### 26.7.2.1. Major Upgrades

With ZFS, boot environments allow an upgrade to be performed without interrupting running software. If the system does not use root-on-ZFS: backup before upgrading.

##### 26.7.2.1.1. Preparations

Change `/usr/local/etc/pkg/repos/FreeBSD-base.conf` to target the correct major release like `base_release_0` for 15.0-RELEASE, to look like this:



This file might change to be included in RELEASE with a different path.

```
FreeBSD-base {
    url = "pkg+https://pkg.freebsd.org/${ABI}/base_release_0";
    mirror_type = "srv";
    signature_type = "fingerprints";
    fingerprints = "/usr/share/keys/pkg";
    enabled = yes;
```

```
}
```



As pkgbase with FreeBSD 14 is experimental, so is a pkgbase major upgrade to 15. At the time of this writing in some edge-cases the major upgrade removes pkg and therefore segfaults. This is considered a (known issue) for 15.0-RELEASE. To work around this issue, lock pkg before upgrading.

```
# pkg -c /mnt/upgrade lock pkg
```

An experimental major upgrade from FreeBSD 14, to 15 (or 16.0-CURRENT), should be followed by installation of meta package `FreeBSD-set-minimal`.

Save a list of the non-base packages in case they are needed later:

```
pkg prime-origins | sort -u > /var/tmp/pkg-prime-origins.txt
```

#### 26.7.2.1.2. major upgrade with zfs

Create a boot environment using bectl(8) and name it according to the version upgrading to

```
# bectl create 15.0-RELEASE
```

Create a directory then mount the new boot environment there to enable working with pkg chroot(8) and rootdir.

```
# mkdir /mnt/upgrade
# bectl mount 15.0-RELEASE /mnt/upgrade
```

The next step will upgrade the boot environment to the specified version.



This step might remove non-base packages, which could include the running desktop environment.

Set the environment variable ABI to upgrade the major version (replace amd64 with the architecture and 15 with the targeted version). Set the pkg-static chroot directory to the boot environment mountpoint.

```
# env ABI=FreeBSD:15:amd64 pkg-static -c /mnt/upgrade upgrade -r FreeBSD-base
```

There will be a prompt asking about ignoring the version mismatch looking like this:

```
Newer FreeBSD version for package FreeBSD-zoneinfo:
To ignore this error set IGNORE_OSVERSION=yes
```

```
- package: 1500058
- running userland: 1500000
Ignore the mismatch and continue? [y/N]:
```

Check and confirm that.

To check if that was successful, chroot into the mountpoint of the boot environment and run `freebsd-version -ku`.

```
# chroot /mnt/upgrade
# freebsd-version -ku
# exit
```

Activate the boot environment temporarily and reboot.

```
# bectl activate -t 15.0-RELEASE
# shutdown -r now
```

If the system does not work as expected: restart the OS to cease using the temporarily active boot environment. The next boot will reuse the pre-upgrade environment.

If the system runs stable after reboot, don't forget to permanently activate this boot environment to prevent accidentally booting the old system.

```
bectl activate 15.0-RELEASE
```

##### 26.7.2.1.3. major upgrade without zfs

Set the environment variable ABI to upgrade the major version (replace amd64 with the architecture and 15 with the targeted version).

```
# env ABI=FreeBSD:15:amd64 pkg-static upgrade -r FreeBSD-base
```

There will be a prompt to ask about ignoring the version mismatch looking like this:

```
Newer FreeBSD version for package FreeBSD-zoneinfo:
To ignore this error set IGNORE_OSVERSION=yes
- package: 1500058
- running userland: 1500000
Ignore the mismatch and continue? [y/N]:
```

Check and confirm that.

To check if the upgrade was successful, run `freebsd-version -ku`.

Then reboot.

#### 26.7.2.1.4. Post-upgrade tasks

If pkg was locked during the update as a workaround remove the lock like this:

```
# pkg unlock pkg
```

After upgrading to a new major version, updates and upgrades of installed packages to match the ABI version may be necessary.

```
# pkg upgrade
```

Consider getting help from FreeBSD Support when experiencing issues.

### 26.7.3. Manually building pkgbase and publishing it to the local network

When building custom pkgbase packages clone the FreeBSD source tree:

```
# cd /usr/src
# git clone https://github.com/freebsd/freebsd-src.git /usr/src
```

Check out the branch for the release to build packages for:

```
# git checkout releng/14.3
```

Start the building process, depending on the resources available this could take some while. Set the parallel processes according to the CPU core count.

This example is written for an 8 core CPU:

```
# make -j8 buildworld && make -j8 buildkernel && make -j8 packages
```

When building frequently, consider using devel/ccache to speed up subsequent builds from the cache.

After building, the packages will get saved into `/usr/obj/usr/src/repo/FreeBSD:14:amd64/14.3p2` or something like that, depending on the version build.

To publish these packages to the network set up a nginx service and use this location in the http server configuration:

```
location /FreeBSD:14:amd64 {
  alias /usr/obj/usr/src/repo/FreeBSD:14:amd64/;
```

```
    autoindex on;
  }
```

And reload the nginx service.

When not using https, use a small configuration file on the clients to target the pkgbase version just built by editing `/usr/local/etc/pkg/repos/upgrade.conf`:

```
upgrade: {
    url          = http://ip.of.the.server/FreeBSD:14:amd64/14.3p2
    enabled      = yes
}
```

and use it as written above (but use -r upgrade instead of FreeBSD-base).

## 26.8. Tracking for Multiple Machines

When multiple machines need to track the same source tree, it is a waste of disk space, network bandwidth, and CPU cycles to have each system download the sources and rebuild everything. The solution is to have one machine do most of the work, while the rest of the machines mount that work via NFS. This section outlines a method of doing so. For more information about using NFS, refer to Network File System (NFS).

First, identify a set of machines which will run the same set of binaries, known as a *build set*. Each machine can have a custom kernel, but will run the same userland binaries. From that set, choose a machine to be the *build machine* that the world and kernel are built on. Ideally, this is a fast machine that has sufficient spare CPU to run `make buildworld` and `make buildkernel`.

Select a machine to be the *test machine*, which will test software updates before they are put into production. This *must* be a machine that can afford to be down for an extended period of time. It can be the build machine, but need not be.

All the machines in this build set need to mount /usr/obj and /usr/src from the build machine via NFS. For multiple build sets, /usr/src should be on one build machine, and NFS mounted on the rest.

Ensure that /etc/make.conf and /etc/src.conf on all the machines in the build set agree with the build machine. That means that the build machine must build all the parts of the base system that any machine in the build set is going to install. Also, each build machine should have its kernel name set with `KERNCONF` in /etc/make.conf, and the build machine should list them all in its `KERNCONF`, listing its own kernel first. The build machine must have the kernel configuration files for each machine in its /usr/src/sys/arch/conf.

On the build machine, build the kernel and world as described in Updating FreeBSD from Source, but do not install anything on the build machine. Instead, install the built kernel on the test machine. On the test machine, mount /usr/src and /usr/obj via NFS. Then, run `shutdown now` to go to single-user mode in order to install the new kernel and world and run `etcupdate` as usual. When done, reboot to return to normal multi-user operations.